

RAG-pohjainen kyberuhkatiedon hakujärjestelmä

Sami Rautén

Metropolia Ammattikorkeakoulu

Insinööri (AMK)

Tieto- ja viestintätekniikka

Insinöörityö

1.5.2025

Tiivistelmä

Tekijä:	Sami Rautén
Otsikko:	RAG-pohjainen kyberuhkatiedon hakujärjestelmä
Sivumäärä:	51 sivua + 3 liitettä
Aika:	1.5.2025

Tutkinto:	Insinööri (AMK)
Tutkinto-ohjelma:	Tieto- ja viestintätekniikka
Ohjaajat:	Janne Salonen, Tommi Hytönen

Tässä insinöörityössä kehitettiin ja arvioitiin järjestelmä, joka hyödyntää Retrieval-Augmented Generation (RAG) -arkkitehtuuria kyberuhkatiedon käsittelyssä. Tavoitteena oli rakentaa kokonaisuus, joka kykenee keräämään uhkatietoa avoimista lähteistä, muuntamaan sen vektoripohjaiseen muotoon ja tuottamaan lähteistettyjä vastauksia suurten kielimallien avulla. Toteutuksessa yhdistettiin OpenCTI-alustan tarjoama tiedonkeruu, Elasticsearchin vektorihaku, semanttinen fallback-mekanismi sekä kielimallien (LLM) ohjattu generointi.

Järjestelmä suunniteltiin modulaariseksi ja kontitettiin Docker-ympäristöön. Käytännön implementaatiossa käytettiin yli 600 000 uhkatiedon dokumenttia, joista generoitiin embeddingit BAAI:n bge-large-en-v1.5-mallilla. Hakuprosessi toteutettiin hybridimallilla, joka yhdistää vektoripohjaisen semanttisen haun ja perinteisen avainsanahaun. Generointivaiheessa käytettiin dynaamisia kehotteita, joilla ohjattiin kielimallia muodostamaan lähteisiin perustuvia vastauksia. Käyttöliittymä toteutettiin Streamlitillä.

Työn lopputuloksena havaittiin, että RAG-pohjainen järjestelmä soveltuu erityisesti strategisen ja operatiivisen kyberuhkatiedustelun tueksi, jossa dokumentaatio on jo jäsennelty ja kysymykset liittyvät laajempiin kokonaisuuksiin. Haasteita ilmeni yksittäisten IoC-tietojen käsittelyssä, datan epäyhtenäisyydessä sekä kielimallien kontekstirajoitteissa. Näihin kehitettiin ratkaisumalleja muun muassa chunkingilla, fallback-haulla ja käyttöliittymän säätöparametreilla.

Järjestelmä osoitti, että RAG-menetelmää voidaan hyödyntää tehokkaasti kyberuhkatiedon rikastamisessa ja jäsentämisessä, kunhan sen käyttö rajoitetaan oikeisiin tilanteisiin ja se liitetään osaksi laajempaa analyysiprosessia. Työ tarjoaa pohjan jatkokehitykselle esimerkiksi agenttisten arkkitehtuurien, laajennettujen konteksti-ikkunoiden ja automatisoidun tiedonhankinnan elinkaarimallin kehitykseen.

Avainsanat:	Kyberuhkatiedustelu, Retrieval-Augmented Generation, RAG, kielimallit, semanttinen haku, OpenCTI, Elasticsearch, LLM, automaatio, CTI-järjestelmä, uhkatiedon esikäsittely

Tämän opinnäytetyön alkuperä on tarkastettu Turnitin Originality Check -ohjelmalla.

Abstract

Author:	Sami Rautén
Title:	RAG-based Cyber Threat Intelligence Retrieval System
Number of Pages:	51 pages + 3 appendices
Date:	1 May 2025
Degree:	Bachelor of Engineering
Degree Programme:	Information and Communications Technology
Supervisors:	Janne Salonen, Tommi Hytönen

This thesis presents the design and implementation of a threat intelligence retrieval system based on the Retrieval-Augmented Generation (RAG) architecture. The system integrates automated data collection, preprocessing, semantic search, and large language model (LLM)-based response generation to support cyber threat intelligence (CTI) workflows. A key objective was to enable efficient utilization of public threat data by converting heterogeneous sources into a unified and searchable format.

The technical implementation consists of five core components: data ingestion via OpenCTI, document preprocessing and vectorization using BAAI's embedding model, hybrid search using Elasticsearch, prompt generation for LLMs, and a user interface developed with Streamlit. The system was capable of returning context-aware and source-referenced responses, particularly when the input data was structured and relevant to operational or strategic analysis.

However, several limitations were observed, such as hallucinations caused by ambiguous prompts, challenges in handling standalone indicators of compromise (IoCs), and the short operational lifespan of certain threat data. The system functioned best when combining automated analysis with expert validation, confirming that RAG-based methods are most effective when used as part of a broader CTI architecture.

Keywords: Cyber Threat Intelligence, Retrieval-Augmented Generation, RAG, Large Language Models, Semantic Search, OpenCTI, Elasticsearch, Automation, CTI System, Threat Data Processing

Sisällys

Lyhenteet ja käsitteet

1 Johdanto....1

2 Tutkimusasetelma ja tekninen lähestymistapa....2

2.1 Aikaisempi tutkimustieto ja työn rajaus....3
2.2 Tutkimuksen toteutus ja menetelmät....3
2.3 Tutkimuskysymykset....4

3 Teoreettinen viitekehys....5

3.1 Kyberuhkatiedustelun ja automaation perusteet....5
3.1.1 Kyberuhkatiedon ja sen automaation määritelmä....5
3.1.2 Tiedustelusykli ja uhkatiedon elinkaari....8
3.1.3 Nykykäytänteet....11
3.2 Tekoäly ja koneoppiminen kyberuhkatiedon automaatiossa....13
3.2.1 Kielimallit, vektorihaku ja tiedon esikäsittely....14
3.2.2 Tekoälyn hyödyntäminen tiedustelusyklissä....15
3.3 Retrieval-Augmented Generation -arkkitehtuuri (RAG)....16
3.3.1 Mitä on Retrieval-Augmented Generation?....17
3.3.2 Kielimallit osana uhkatiedon analyysiä ja RAG-menetelmää….18
3.3.3 Aiemmat tutkimukset RAG-menetelmän käytöstä kyberturvallisuudessa....19

4 Järjestelmän suunnittelu ja toteutus....20

4.1 Arkkitehtuurin yleiskuvaus....20
4.2 Tiedonkeruu ja lähteet....22
4.3 Tiedon esikäsittely ja vektorointi....24
4.4 Semanttinen haku ja fallback....27
4.5 Promptin rakenne ja kielimallien hyödyntäminen....30
4.6 Käyttöliittymän toteutus....32
4.7 Testaus: Kysymyksen muodostaminen ja vastausten arviointi....37
4.7.1 Testijärjestelyt ja järjestelmän toiminta testauksessa....37

4.7.2 Testitapaus ja tulokset....38
4.7.3 Analyysi ja laadullinen tulos....39
4.8 Teknisiä haasteita ja ratkaisumalleja....41
4.9 Yhteenveto toteutuksesta....42

5 Johtopäätökset....43

Lähteet....47

Liitteet
Liite 1: Järjestelmän konttikonfiguraatio (docker-compose.yml)
Liite 2: Kehotteen muodostus ja haku (prompt_template.py ja retrieval.py)
Liite 3: Cyber Threat Intelligence RAG Assistant -kysely

Lyhenteet ja käsitteet

AI (Artificial Intelligence)
Keinoäly. Viittaa järjestelmiin, jotka kykenevät suorittamaan älykkäitä toimintoja, kuten päättelyä, oppimista tai ongelmanratkaisua.

CTI (Cyber Threat Intelligence)
Kyberuhkatiedustelu. Tiedonkeruu- ja analyysiprosessi, jossa kootaan, jäsennetään ja tulkitaan kyberuhkiin liittyvää tietoa päätöksenteon tueksi.

IoC (Indicator of Compromise)
Indikaattori vaarantumisesta. Tekninen havainto (esim. IP-osoite, domain tai tiedoston hash), joka viittaa mahdolliseen tietoturvaloukkaukseen.

LLM (Large Language Model)
Suuri kielimalli. Koneoppimiseen perustuva malli, joka kykenee käsittelemään ja tuottamaan luonnollista kieltä laajasta tekstimateriaalista oppineena.

OpenCTI (Open Cyber Threat Intelligence)
Avoimen lähdekoodin alusta kyberuhkatiedon kokoamiseen, hallintaan ja jakamiseen STIX 2.1 -formaatissa.

RAG (Retrieval-Augmented Generation)
Hakuun perustuva generointi. Kielimallien arkkitehtuuri, jossa ulkoisesta tietovarastosta haettua sisältöä yhdistetään kehotteeseen (prompt) ennen vastausta.

RSS (Really Simple Syndication)
Web-syötemuoto, jota käytetään uutisvirtojen ja päivitysten automaattiseen jakamiseen.

STIX 2.1 (Structured Threat Information Expression)
Standardoitu formaatti, jolla kyberuhkatietoa voidaan mallintaa koneellisesti käsiteltävässä muodossa.

TTP (Tactics, Techniques and Procedures)
Taktiikat, tekniikat ja menetelmät. Yhteinen kuvaustapa hyökkääjien toiminnalle, erityisesti MITRE ATT&CK -viitekehyksessä.

Vector Search
Vektorihaku. Hakumenetelmä, jossa kysely ja dokumentit muunnetaan numeerisiin esityksiin (vektoreiksi) ja samankaltaisuus lasketaan esimerkiksi kosinietäisyyden avulla.

Fallback-haku
Varahakumenetelmä, joka käynnistetään, jos semanttinen haku ei tuota riittävästi tuloksia. Työssä toteutettu avainsanoihin perustuvalla match-haulla.

Prompt (Kehote)
Kielimallille esitettävä syöte, joka määrittää sen tuottaman vastauksen rakenteen, sisällön ja tyyliin liittyvät reunaehdot.

Token
Kielimallin käsittelemä yksikkö, joka voi olla sana, sanan osa tai merkki. Mallin kontekstikapasiteetti mitataan yleensä maksimitoken-määränä (esim. 4000 tokenia).

1 Johdanto

Kyberuhkatiedon (Cyber Threat Intelligence, CTI) merkitys on kasvanut nopeasti organisaatioiden turvallisuudenhallinnassa, kun kyberuhkien määrä ja monimuotoisuus ovat lisääntyneet. Ajantasainen ja luotettava uhkatieto on välttämätöntä, jotta organisaatiot voivat ennakoida, havaita ja torjua uhkia tehokkaasti. Perinteiset menetelmät uhkatiedon hallinnassa ovat kuitenkin osoittautuneet riittämättömiksi käsittelemään suuria tietomääriä ja nopeasti muuttuvaa uhkaympäristöä. Manuaaliset analyysit ja avainsanoihin perustuvat haut eivät enää kykene tunnistamaan uusien uhkien monimutkaisuutta tai kontekstia riittävän kattavasti.

Viime vuosina tekoälyn ja erityisesti koneoppimisen kehittyminen on mahdollistanut älykkäämpien ja automaattisempien järjestelmien rakentamisen. Suurten kielimallien (Large Language Models, LLM) kehitys on tuonut uudenlaisen kyvyn ymmärtää kieltä syvällisemmin ja samalla mahdollistaa hakutoiminnot, jotka perustuvat sanojen merkityksiin ja laajempaan kontekstiin pelkkien avainsanojen sijasta. Näiden teknologioiden avulla tiedon kerääminen, analysointi ja hyödyntäminen voidaan toteuttaa aiempaa tehokkaammin ja automaattisemmin kyberturvallisuuden alalla.

Tämän insinöörityön tavoitteena oli kehittää Proof-of-Concept (PoC) -tasoinen ratkaisu, jossa tutkitaan Retrieval-Augmented Generation (RAG) -menetelmän soveltuvuutta kyberuhkatiedon automaattisessa käsittelyssä ja analysoinnissa. Työssä tarkastellaan, kuinka avoimista lähteistä (Open Source Intelligence, OSINT) kerätty uhkatieto voidaan esikäsitellä, muuntaa koneellisesti hyödynnettävään muotoon sekä hakea semanttisesti hyödyntäen suuria kielimalleja, vektorihakua ja hybridihakumenetelmiä, kuten fallback-mekanismeja.

Työn päätutkimuskysymyksenä on selvittää, voidaanko RAG-menetelmää hyödyntää tehokkaasti kyberuhkatiedon käsittelyssä ja analysoinnissa Proof-of-Concept-tasoisessa järjestelmässä. Tarkemmat tutkimuskysymykset ja rajaukset esitetään luvussa 2.

Työn rakenne etenee seuraavasti: Luvussa 2 kuvataan tutkimusasetelma, aikaisempi tutkimustieto ja työn rajaukset. Luku 3 esittelee teoreettisen viiteke-hyksen, jossa tarkastellaan tekoälyn, koneoppimisen, kielimallien ja RAG-arkkitehtuurin keskeisiä käsitteitä. Luvussa 4 kuvataan tekninen toteutus ja järjestelmän kehittämisen valinnat. Luvussa 5 analysoidaan toteutetun järjestelmän toimivuutta ja saatuja tuloksia sekä esitetään johtopäätökset ja jatkokehitysehdotukset.

2 Tutkimusasetelma ja tekninen lähestymistapa

Tämän insinöörityön tavoitteena oli kehittää Proof-of-Concept (PoC) -tasoinen järjestelmä, jolla tutkitaan Retrieval-Augmented Generation (RAG) -menetelmän soveltuvuutta kyberuhkatiedon automaattisessa käsittelyssä ja analysoinnissa. Työssä suunniteltiin ja rakennettiin tekninen ratkaisu, joka perustuu suurten kielimallien, vektoripohjaisen semanttisen haun ja hybridihakumenetelmien (fallback-mekanismit) käyttöön. Ratkaisun merkitys kyberuhkatiedustelun ammatillisessa kontekstissa on merkittävä, sillä nykyiset manuaaliset ja perinteiset menetelmät eivät riitä reagoimaan nopeasti muuttuviin uhkiin.

Tutkimus toteutettiin konstruktiivisena kehittämistutkimuksena, joka edellytti iteratiivista kehitysprosessia, komponenttipohjaista suunnittelua sekä järjestelmällistä dokumentointia ja testausta. Tutkimuksen eettiset periaatteet huomioitiin erityisesti tiedonkeruun, käsittelyn ja analysoinnin läpinäkyvyydessä sekä käytettyjen lähteiden asianmukaisessa ja luotettavassa hyödyntämisessä.

2.1 Aikaisempi tutkimustieto ja työn rajaus

Aikaisempi tutkimus kyberuhkatiedon alalla on osoittanut, että perinteiset sääntöihin ja avainsanoihin perustuvat hakumenetelmät eivät riitä dynaamisessa ja monimuotoisessa uhkaympäristössä. Aiemmat ratkaisut ovat kärsineet erityisesti tiedon semanttisen merkityksen puutteellisesta ymmärtämisestä sekä hitaasta reagointikyvystä uusiin uhkiin.

Tässä työssä rajattiin seuraavat asiat tutkimuksen ulkopuolelle:

- perinteiset avainsana- tai sääntöpohjaiset hakumenetelmät
- tuotantotason suorituskyvyn, skaalautuvuuden ja tietoturvaominaisuuksien optimointi
- kaupalliset valmiit ratkaisut (Threat Intelligence Platforms)
- muiden kuin RAG-menetelmän tarkempi vertailu.

Työn pääpaino oli kehittää ja tutkia PoC-tason järjestelmää, joka yhdistää semanttisen tiedonhaun ja automaattisen analyysin kyberuhkatiedon alalla.

2.2 Tutkimuksen toteutus ja menetelmät

Tutkimuksessa hyödynnettiin konstruktiivista kehittämistutkimusta, jonka tavoitteena oli rakentaa tekninen ratkaisu ja osoittaa sen toimivuus käytännössä. Ratkaisu kehitettiin ketterien ohjelmistokehitysperiaatteiden, iteratiivisen prototyyppikehityksen ja systemaattisen testauksen avulla, mikä varmisti korkean luotettavuuden ja toistettavuuden.

Tutkimuksessa käytettiin ensisijaisesti avoimista lähteistä (Open Source Intelligence, OSINT) kerättyä uhkatietoa. Tiedon esikäsittelyssä hyödynnettiin kieli-

mallien vaatimaa vektorointia, jonka jälkeen toteutettiin semanttinen haku käyttäen RAG-menetelmää. Lisäksi rakennettiin fallback-mekanismi, joka paransi järjestelmän suorituskykyä ja tiedon relevanssia tilanteissa, joissa ensisijainen hakumenetelmä ei tuottanut riittäviä tuloksia.

2.3 Tutkimuskysymykset

Tutkimuksen päätutkimuskysymys oli,

Voidaanko Retrieval-Augmented Generation (RAG) -menetelmää hyödyntää tehokkaasti kyberuhkatiedustelussa automatisoimalla avoimista lähteistä kerätyn uhkatiedon esikäsittely, semanttinen haku ja analyysi suurten kielimallien, vektorihakujen ja fallback-mekanismien avulla Proof-of-Concept-tason järjestelmässä?

Tätä pääkysymystä tarkennettiin seuraavilla alakysymyksillä:

1. Millaista uhkatietoa voidaan automaattisesti kerätä ja jäsentää avoimista lähteistä hyödyntämällä suuria kielimalleja ja semanttista hakua?

2. Miten tiedon esikäsittely ja vektorointi vaikuttavat kielimallien kykyyn tulkita ja käsitellä uhkatietoa?

3. Missä määrin hybridihakumenetelmät (erityisesti fallback-mekanismit) parantavat tiedon saatavuutta ja relevanssia epävarmoissa hakutilanteissa?

4. Kuinka hyvin automaattisesti tuotettu uhkatieto kykenee tukemaan kyberturvallisuuden asiantuntijatyötä käytännön analyysissä?

Näihin kysymyksiin vastaamalla pyrittiin luomaan käytännössä hyödyllinen, luotettava ja sovellettavissa oleva ratkaisu, jonka avulla voidaan tehokkaasti parantaa kyberuhkatiedon hallintaa ja analysointia.

3 Teoreettinen viitekehys

3.1 Kyberuhkatiedustelun ja automaation perusteet

Kyberuhkatiedustelu on keskeinen osa organisaatioiden kyberturvallisuusstrategiaa ja sen automatisointi on muuttunut välttämättömäksi tehokkaan ja ajantasaisen uhkatiedon hallinnassa. Tässä luvussa käsitellään kyberuhkatiedon ja sen automaation määritelmiä, toimintaa ohjaavaa tiedustelusykliä sekä nykyisiä käytäntöjä ja teknologioita, jotka mahdollistavat tiedon keräämisen, rikastamisen ja analysoinnin automaattisesti. Tavoitteena on muodostaa kokonaiskuva siitä, miten uhkatiedon arvoa voidaan maksimoida ja asiantuntijoiden työtä tehostaa modernien automaatiomenetelmien avulla.

3.1.1 Kyberuhkatiedon ja sen automaation määritelmä

Kyberuhkatiedustelu on järjestelmällinen prosessi, jonka tavoitteena on tuottaa tietoa kyberuhkista päätöksenteon, suojaustoimenpiteiden ja tilannetietoisuuden tueksi. Uhkatieto määritellään laajasti: se kattaa kaiken tiedon, joka auttaa organisaatiota tunnistamaan, arvioimaan, ennakoimaan, seuraamaan tai reagoimaan kyberuhkiin. Tällaisia tietotyyppejä ovat esimerkiksi hyökkäysindikaattorit (IoC), hyökkäysten taktiikat, tekniikat ja prosessit (TTP), hälytykset, uhkaraportit sekä suojausjärjestelmien suositellut konfiguraatiot. Organisaatiot tuottavat uhkatietoa jo osana omia tietoturva- ja valvontatoimintojaan, mutta sen järjestelmällinen hyödyntäminen ja jakaminen jää usein puutteelliseksi. [1, s. 3-9.]

Kyberuhkatiedustelu kattaa sekä reaktiivisen että ennakoivan toiminnan. Se ei rajoitu pelkästään toteutuneiden hyökkäysten jälkikäsittelyyn, vaan pyrkii myös havaitsemaan ja ehkäisemään uhkia ennen niiden toteutumista. [2.] Uhkatieto jaetaan yleensä kolmeen tasoon: strateginen, operatiivinen ja taktinen. Tämä jako, joka juontaa juurensa eri valtioiden asevoimien tiedustelukäytännöistä, on vakiintunut myös viranomaisten ja kyberalan toimijoiden keskuudessa. [3; 4; 5.]

- **Strateginen taso** tarkastelee uhkatoimijoiden motiiveja, kyvykkyyksiä ja vaikutuksia yhteiskuntaan tai liiketoimintaan.

- **Operatiivinen taso** keskittyy uhkatoiminnan toteutustapoihin, esimerkiksi hyökkääjien käyttämien taktiikoiden ja tekniikoiden tarkasteluun hyödyntäen viitekehyksiä kuten MITRE ATT&CK.

- **Taktinen taso** käsittelee konkreettisia teknisiä havaintoja, kuten haittaohjelmien tiivisteitä, IP-osoitteita ja verkkotunnuksia.



Kuva 2. Uhkatiedon analyysitasot: strateginen, operatiivinen ja taktinen. [3; 4; 5; 6.]

Kuva 2 havainnollistaa hierarkkisen rakenteen ja erityispiirteet, joita eri tasojen analyysissä tulee huomioida.

Automatisoidussa uhkatiedustelussa painottuu erityisesti taktisen tason tiedon hallinta. Koska tiedon luotettavuus voi vaihdella, NIST korostaa rajoitetun attribuoinnin ilmiötä (limiting attribution): tekniset todisteet voivat olla harhaanjohta-

via, manipuloituja tai anonymisoituja. Automaation roolin tulee ulottua tiedon keräämisestä myös sen jalostamiseen ja kontekstualisointiin. Pelkkä raakadata voi johtaa harhaan, jos sitä ei ole rikastettu ja arvioitu. [1, s. 5-6.]

Rikastettu tieto eli matured knowledge muodostuu, kun yksittäisiä havaintoja analysoidaan ja liitetään laajempiin kokonaisuuksiin, kuten uhkakampanjoihin tai toimijoiden käyttämiin taktiikoihin, tekniikoihin ja prosesseihin (TTP) [1, s. 4-6.] Rikastaminen parantaa analyysin tarkkuutta ja syventää kokonaisymmärrystä uhkatoimijoiden toiminnasta.

Uhkatiedon analyysissä epävarmuustekijöitä käsitellään todennäköisyyden (probability) ja luottamuksen (confidence) käsitteiden avulla. Heuerin mukaan todennäköisyys arvioi, kuinka todennäköisesti analyysin tulokset pitävät paikkansa, ja luottamus kuvaa analyytikon käsitystä lähteiden ja analyysin luotettavuudesta. [7, s. 143-145, 155-157.]

NIST:n riskienhallinnan viitekehys (Risk Management Framework, RMF) korostaa, että riskien arvioinnissa tulee huomioida uhkatapahtumien todennäköisyys, vaikutukset sekä hyökkääjän kyvykkyys ja kohdentaminen [8, s. 5-12]. Riskimallien dokumentointi varmistaa analyysien toistettavuuden ja vertailtavuuden.

Tässä työssä keskitytään erityisesti taktisen tason uhkatiedon automaattiseen keräämiseen ja käsittelyyn avoimista lähteistä (Open Source Intelligence, OSINT), kuten RSS-syötteistä ja AlienVault OTX:sta. RSS-syötteet ovat edelleen relevantteja rakenteisuutensa, ajastettavuutensa ja koneystävällisyytensä vuoksi [9, s. 4-6].

Automaatiolla tarkoitetaan tässä työssä toistuvien ja rakenteeltaan ennakoitavien toimintojen ohjelmallista suorittamista, kuten tiedonkeruuta, jäsentelyä, indikaattorien tunnistamista ja tallentamista. Vaikka automaatio vähentää manuaalista työtä, asiantuntijan rooli tiedon arvioinnissa ja päätöksenteossa säilyy keskeisenä. [2.]

3.1.2 Tiedustelusykli ja uhkatiedon elinkaari

Kyberuhkatiedustelun toimintaa voidaan kuvata tiedustelusyklin (intelligence cycle) avulla. Tiedustelusykli esittää prosessin vaiheittaisena ja jatkuvana, mahdollistaen tiedon keräämisen, jalostamisen ja jakamisen päätöksenteon tueksi. Tässä työssä sovelletaan viisivaiheista mallia, joka on laajasti käytössä kybertoimialalla ja yhteensopiva kansainvälisten suositusten kanssa. [10; 11.]

Viisivaiheinen tiedustelusykli koostuu seuraavista vaiheista [10; 11.]:

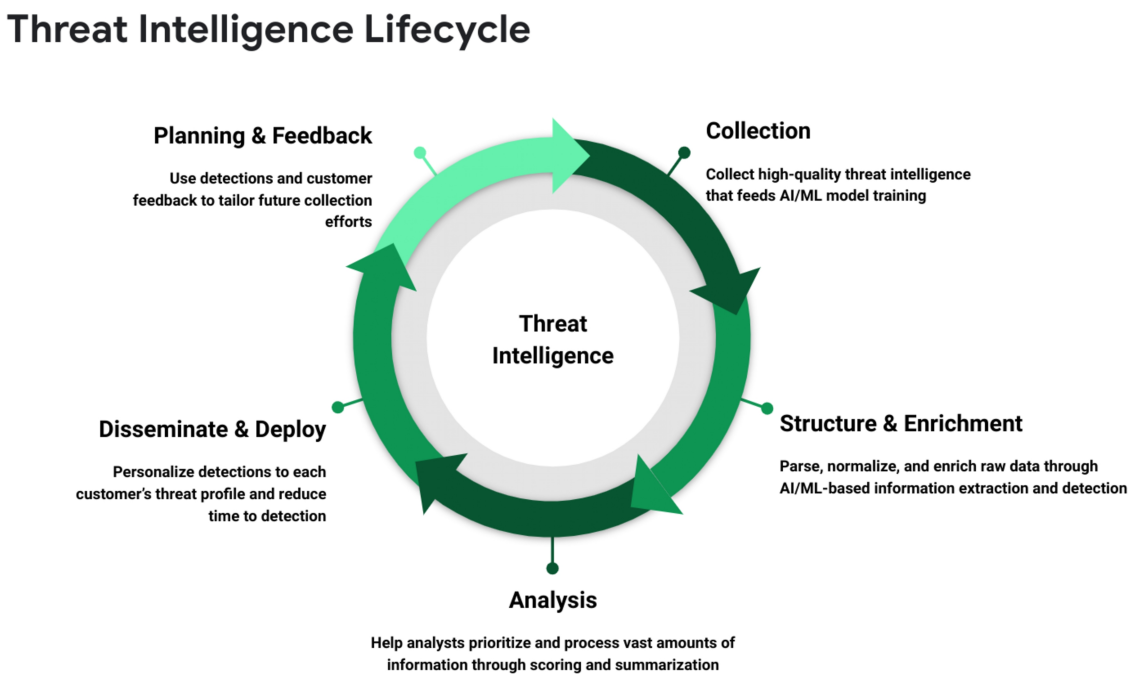
- **Suunnittelu ja palaute**: Tiedustelutoiminnan ohjaaminen tavoitteiden mukaisesti sekä palautteen hyödyntäminen keruuprosessin parantamiseksi.

- **Tiedonhankinta**: Korkealaatuisen uhkatiedon kerääminen avoimista ja suljetuista lähteistä, kuten RSS-syötteistä ja keskustelufoorumeista.

- **Prosessointi**: Raakadatan muuntaminen rakenteiseksi ja analysoitavaksi tiedoksi, mukaan lukien tekoälyn ja koneoppimisen hyödyntäminen tiedon rikastamisessa.

- **Analyysi ja jalostaminen**: Tiedon syvällinen tarkastelu, uhkatrendien tunnistaminen ja johtopäätösten muodostaminen operatiivisen päätöksenteon tueksi.

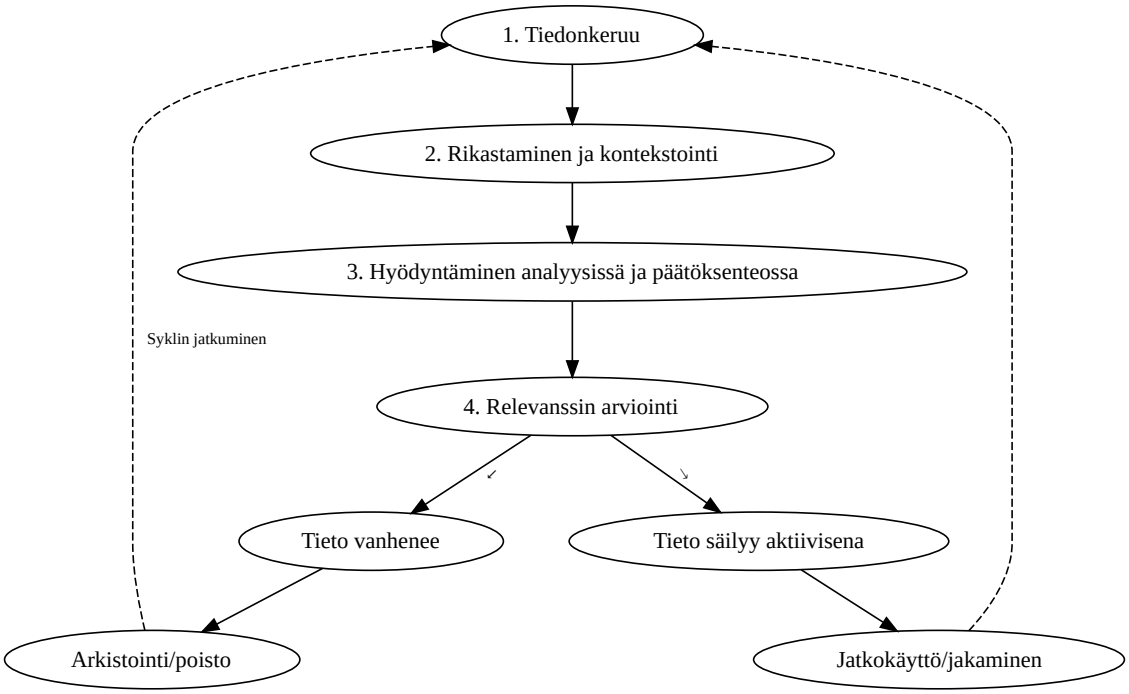
- **Jakaminen**: Analysoidun tiedon toimittaminen oikeille vastaanottajille, organisaation sisällä ja yhteistyötahoille.



Kuva 3: Kyberuhkatiedustelun syklinen malli Mandiantin (Google) mukaan. [10.]

Kuva 3 havainnollistaa yleisen tiedustelusyklin, joka mahdollistaa toistettavuuden, laadunvalvonnan ja rakenteellisen lähestymistavan uhkatiedon käsittelyyn riippumatta siitä, tehdäänkö työ manuaalisesti vai automaation tukemana. Automaatioteknologiat voivat merkittävästi tehostaa erityisesti tiedonhankinnan, prosessoinnin ja analyysin alkuvaiheita. Päätöksenteossa ja tiedon jakamisessa asiantuntijan harkinta ja kontekstituntemus säilyvät välttämättöminä. [1; 10; 11.]

Kyberuhkatiedon elinkaari kuvaa, kuinka tiedon arvo ja käyttökelpoisuus muuttuvat ajan myötä ja eri konteksteissa. Tieto ei ole staattista, vaan sen merkitys vaihtelee hyökkääjien toiminnan ja ympäristötekijöiden mukaan. [12; 8.]



Kuva 4. Uhkatiedon elinkaarimalli ja automaation rooli.

Kuvassa 4 esitetään uhkatiedon elinkaarimalli, jossa korostuu automaation merkitys toistuvissa ja ennakoitavissa toiminnoissa, kuten tiedon keräämisessä, alustavassa analyysissä, luokittelussa ja vanhentuneen tiedon poistamisessa. [12; 8.] Esimerkiksi valvontalistoilta voidaan automaattisesti poistaa IP-osoitteita, jotka eivät ole olleet aktiivisia määritellyn ajan kuluessa. Samalla automaatio mahdollistaa rikastetun ja analysoidun tiedon säilyttämisen ja jatkuvan päivittämisen, mikä tukee tiedon käyttöä pitkällä aikavälillä.

Tiedon käyttökelpoisuus ei riipu ainoastaan teknisestä olemassaolosta, vaan ennen kaikkea ajankohtaisuudesta ja kontekstista. ENISA korostaa vuoden 2023 uhkakatsauksessaan [2, s. 3-4], että esimerkiksi IP-osoitteet tai haittaohjelmatunnisteet voivat vanhentua tunneissa, kun taas strategisen tason tieto voi olla käyttökelpoista jopa vuosia. [2; 13; 6.]

NIST kuvaa tiedon kehittämistä termillä "knowledge maturation", eli hajanaisten havaintojen rikastumista analyysin ja yhteistyön kautta kokonaisvaltaiseksi uh-

katiedoksi [1, s. 5-6.] Tärkeää on ymmärtää, että automaatio ei poista ihmisen merkitystä analyysissa: asiantuntija on edelleen välttämätön priorisoimaan, tulkitsemaan ja arvioimaan tiedon luotettavuutta ja kontekstuaalista merkitystä [7, s. 143-145; 14.]

3.1.3 Nykykäytänteet

Kyberuhkatiedustelun (Cyber Threat Intelligence, CTI) automaatio on siirtynyt kokeiluvaiheista kohti arkipäiväistä käyttöä sekä julkisella että yksityisellä sektorilla. Tämä kehitys näkyy sekä kansallisissa kyberturvallisuusstrategioissa että organisaatioiden käytännöissä, joissa automaattinen tiedonkeruu ja analyysi ovat muodostuneet keskeisiksi kyberturvallisuuden tukipilareiksi. [15, s. 8-11; 16, s. 2.]

Erityisesti tekoälypohjaiset ratkaisut, kuten Retrieval-Augmented Generation (RAG), ovat mahdollistaneet osittain tehokkaamman uhkatiedon jalostamisen ja analysoinnin. Ne tarjoavat keinon parantaa organisaatioiden reagointikykyä nopeasti muuttuvissa uhkaympäristöissä. [17, s. 4-5.]

Automaattinen uhkatiedonhallinta perustuu vahvasti avoimiin formaatteihin, kuten STIX- ja TAXII-standardeihin sekä avoimen lähdekoodin alustoihin, kuten MISP ja OpenCTI [18; 16.] Näiden työkalujen avulla uhkatietoa voidaan jäsentää, jakaa ja rikastaa tehokkaasti sekä organisaatioiden sisällä että niiden välillä. Avoimen lähdekoodin ekosysteemit tarjoavat joustavuutta ja mahdollistavat räätälöityjen ratkaisujen kehittämisen, mikä on erityisen tärkeää pienille ja resurssirajoitteisille toimijoille. [15, s. 6-7.]

Käytännössä uhkatiedon keruu tapahtuu lähteistä kuten RSS-syötteet, AlienVault OTX, AbuseIPDB, keskustelufoorumit ja tekniset blogit. Kerätty tieto siirtyy automaatioalustoille pääasiassa REST-rajapintojen kautta. Keruun jälkeen raakadata normalisoidaan ja suodatetaan, minkä jälkeen sitä rikastetaan yhdistä-

mällä esimerkiksi WHOIS-, ASN- ja kampanjatietoon. Rikastusvaihe tukee analyytikoita tarjoamalla ajantasaista ja kontekstualisoitua tietoa, mikä helpottaa reagointia ja päätöksentekoa. [19, s. 5-6; 20, s. 3-4.]

Tekoälypohjaiset ratkaisut, kuten RAG, täydentävät sääntöpohjaisia malleja erityisesti silloin, kun lähdedata on epämuotoista ja monitulkintaista. RAG-arkkitehtuuri mahdollistaa epämuotoisten ja rakenteisten lähteiden yhdistämisen ja tiivistämisen analyytikoille helposti hyödynnettävässä muodossa. [17.] Tekoälyn roolina ei ole korvata asiantuntijaa, vaan tukea ja nopeuttaa tiedon seulontaa ja alustavaa analyysiä [21, s. 11].

Automaation kehittymisestä huolimatta, asiantuntija-analyysi säilyy keskeisenä erityisesti silloin, kun tiedon alkuperä on epävarmaa, kieliversiot (esim. venäjä, kiina) aiheuttavat haasteita tai kun lähteiden luotettavuus vaatii manuaalista arviointia. [19, s. 8; 21, s. 9-11.] Analyytikot suorittavat edelleen manuaalisia tehtäviä, kuten hyökkäysindikaattorien tarkistusta, kampanjoiden ryhmittelyä, priorisointia ja havaintojen validointia useista lähteistä.

Useat kaupalliset ratkaisut, kuten Cortex XSOAR, Falcon XDR ja SYNAPSE, tarjoavat automatisoituja uhkatiedonhallintajärjestelmiä. Näiden järjestelmien heikkoutena on kuitenkin suljettu ekosysteemi ja valmistajariippuvuus, mikä voi rajoittaa räätälöitävyyttä ja kasvattaa kustannuksia. Avoimen lähdekoodin ratkaisut tarjoavat joustavamman ja kustannustehokkaamman vaihtoehdon, erityisesti monitoimijaympäristöissä. [15, s. 6-7; 22, s. 2.]

Nykyaikainen uhkatiedustelujärjestelmä koostuu useista erikoistuneista ja toisiinsa yhteensopivista komponenteista: tiedon kerääminen, normalisointi, rikastus ja esittäminen tapahtuvat hajautetusti mutta yhtenäisessä muodossa. Tavoitteena on tuottaa ajantasaista ja kontekstiin sidottua tilannetietoa, joka tukee nopeaa päätöksentekoa ja reagointia nopeasti muuttuvassa uhkaympäristössä. [20, s. 5; 19, s. 8.]

3.2 Tekoäly ja koneoppiminen kyberuhkatiedon automaatiossa

Tekoäly (AI) viittaa järjestelmiin, jotka kykenevät suorittamaan tehtäviä, joita normaalisti pidettäisiin inhimillistä älyä vaativina, kuten päättelyä, oppimista ja sopeutumista [1.] Koneoppiminen (ML) on tekoälyn osa-alue, jossa järjestelmät oppivat suoraan datasta ilman erillistä ohjelmointia, muodostamalla yleistyksiä havaintojen perusteella. [23.]

Kyberuhkatiedustelussa tekoäly ja koneoppiminen mahdollistavat suurten tietomassojen nopean käsittelyn ja piilevien uhkamallien tunnistamisen, jotka jäisivät huomaamatta perinteisillä sääntöpohjaisilla menetelmillä [23; 2.] Tekoälypohjaiset järjestelmät voivat mukautua uusiin uhkiin ilman erillistä päivitystä, mikä on erityisen tärkeää dynaamisessa uhkaympäristössä.

Koneoppimisen menetelmät voidaan jakaa kolmeen pääluokkaan: valvottuun oppimiseen (esimerkiksi luokittelu tunnetuista uhkamalleista), valvomattomaan oppimiseen (kuten klusterointiin hyökkäyskampanjoiden ryhmittelyssä) ja poikkeamien tunnistamiseen, joka soveltuu uusien, tuntemattomien uhkien havaitsemiseen. [20; 24.] Syväoppiminen, jossa käytetään monikerroksisia neuroverkkoja, mahdollistaa erityisesti rakenteettoman datan, kuten lokitiedostojen ja vapaamuotoisten raporttien, syvällisen analyysin [24.]

Tärkeää on ymmärtää, että tekoäly- ja koneoppimismallit eivät perustu staattisiin sääntöihin, vaan pystyvät mukautumaan jatkuvasti muuttuviin tilanteisiin. Tämä kyky yleistää ja löytää uusia uhkamalleja ilman ennalta määriteltyä kaavaa on yksi tekoälyn keskeisistä eduista kyberuhkatiedustelussa. [1; 25.]

3.2.1 Kielimallit, vektorihaku ja tiedon esikäsittely

Suurten kielimallien (Large Language Models, LLM) kehitys on laajentanut tekoälyn soveltamista kyberuhkatiedusteluun. Foundation-malleiksi kutsutut mallit,

kuten GPT-4, BERT ja Claude, on koulutettu massiivisiin tekstikorpuksiin, ja niitä voidaan hyödyntää erilaisiin tehtäviin pelkällä kontekstin muuttamisella. [26.]

Vektorihaku on keskeinen menetelmä rakenteettoman tiedon käsittelyssä. Sen avulla voidaan löytää semanttisesti samankaltaisia dokumentteja vertaamalla kyselyn ja dokumenttien vektoriesityksiä korkean ulottuvuuden avaruudessa. [27.] Toisin kuin perinteinen avainsanahaku, vektorihaku perustuu merkityssisältöön, ei täsmällisiin sanoihin.

Tokenisointi on olennainen esivaihe sekä kielimallien että vektorimallien toiminnassa. Tokenisoinnissa teksti jaetaan pienemmiksi yksiköiksi, jotka voidaan tehokkaasti muuntaa numeerisiksi vektoreiksi. [28.] Esimerkiksi WordPiece- ja BPE-tokenisaatiomenetelmät mahdollistavat harvinaisten sanojen ja teknisten termien esittämisen siten, että semanttinen informaatio säilyy analyysissa.

Kyberuhkatiedustelussa kielimallit ja vektorihaku tukevat erityisesti tiedonhankinnan ja analyysin vaiheita. Ne mahdollistavat olennaisten entiteettien, kuten haittaohjelmien, TTP-tekniikoiden ja hyökkäysskenaarioiden, tehokkaan erottamisen laajoista tietomassoista. [27; 26.]

3.2.2 Tekoälyn hyödyntäminen tiedustelusyklissä

Kyberuhkatiedustelun tiedustelusykli rakentuu vaiheista, jotka sisältävät suunnittelun ja palautteen, tiedonhankinnan, prosessoinnin, analyysin ja jalostamisen sekä tiedon jakamisen [10; 1.] Tekoälyllä on tärkeä rooli erityisesti tiedonhankinnan, prosessoinnin ja analyysin vaiheissa.

Tiedonhankinnassa tekoäly automatisoi suurten tietomassojen skannauksen ja olennaisen tiedon erottelun esimerkiksi web-scraping-tekniikoilla [10.] Prosessoinnissa koneoppimismallit tukevat datan jäsentämistä ja puhdistamista, jolloin raakadata muunnetaan analysoitavaan muotoon. Analyysivaiheessa syväoppi-

minen ja kielimallit mahdollistavat trendien, poikkeamien ja piilevien yhteyksien tunnistamisen. [24.]

Strategisella tasolla tekoäly auttaa havaitsemaan makrotason muutoksia uhkaympäristössä esimerkiksi semanttisen analyysin ja tekstilouhinnan avulla [25.] Operatiivisella tasolla koneoppiminen tukee hyökkäyskampanjoiden ja TTP-profiilien tunnistamista, mikä parantaa suojaustoimenpiteiden priorisointia [10.] Taktisella tasolla automaatio nopeuttaa indikaattoritiedon keruuta ja validointia, kuten IP-osoitteiden ja haittaohjelmien tiivisteiden käsittelyä. [14.]

Tekoäly tehostaa tiedustelusykliä merkittävästi, mutta se ei korvaa inhimillistä analyysiä. Erityisesti strategisessa ja operatiivisessa analyysissä kontekstin tulkinta ja epävarmuuden hallinta vaativat ihmisen asiantuntemusta. [2; 25.] Tästä syystä parhaat tulokset saavutetaan "human-in-the-loop"-lähestymistavalla, jossa tekoäly tukee asiantuntijaa, mutta ei itsenäisesti tee päätöksiä.

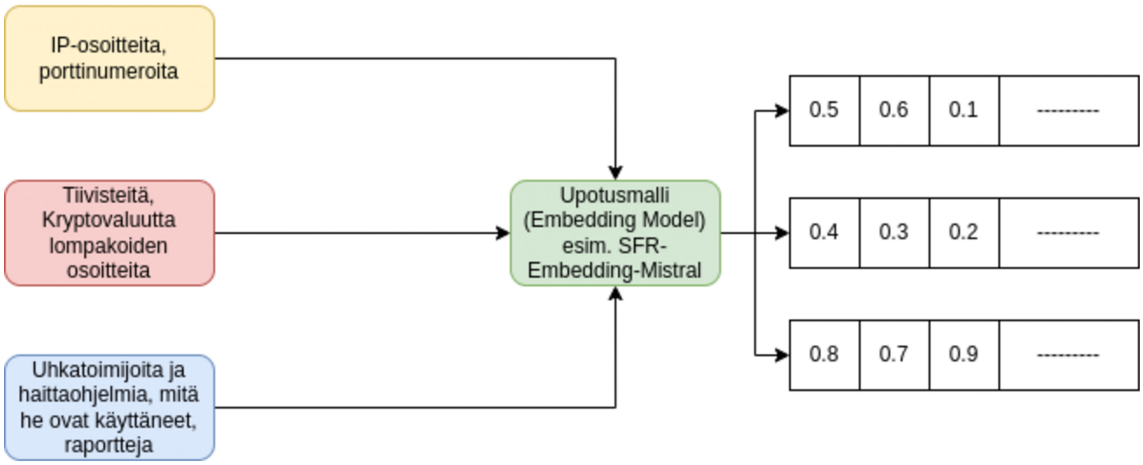
Näitä teknologioita hyödyntäen voidaan rakentaa käytännön järjestelmiä, joissa semanttinen haku ja luonnollisen kielen analyysi yhdistyvät. Tässä työssä vektorihaku toteutettiin Elasticsearch-alustan avulla. Elasticsearch mahdollistaa sekä perinteisen avainsanapohjaisen haun että vektoripohjaisen semanttisen haun yhdistämisen samaan kyselyyn (hybridihaku), mikä parantaa olennaisesti haun tarkkuutta ja merkityslähtöisyyttä. [27; 29.]

Edellä kuvatut menetelmät – kielimallit, tokenisointi, upotusmallit ja semanttinen haku, muodostavat teoreettisen perustan uhkatiedon automaattiselle käsittelylle. Näiden pohjalta siirrytään seuraavaksi tarkastelemaan Retrieval-Augmented Generation (RAG) -arkkitehtuuria, jossa yhdistyvät semanttisen haun ja suurten kielimallien kyvykkyydet vastatakseen uhkatiedon analyysin erityisiin haasteisiin.

3.3 Retrieval-Augmented Generation -arkkitehtuuri (RAG)

Retrieval-Augmented Generation (RAG) yhdistää tiedonhaun ja luonnollisen kielen generoinnin. Malli koostuu kahdesta pääkomponentista: hakumoduulista, joka etsii kysymykseen liittyviä tietoja ulkoisesta lähteestä, ja generointimoduulista, joka muodostaa vastauksen hyödyntäen haettua sisältöä. [31, s. 2-3.]

Toisin kuin perinteiset kielimallit, jotka vastaavat vain koulutusdataansa pohjautuen, RAG hyödyntää ajankohtaista ja ulkoista tietoa vastauksen tukena. Näin voidaan vähentää virheellisiä väittämiä ja parantaa vastausten ajankohtaisuutta ja luotettavuutta. [29.]
Arkkitehtuuri yhdistää mallin sisäisen tietämyksen (parametrinen muisti) ja ulkoisen tietovaraston (ei-parametrinen muisti) saumattomasti [31, s. 1-2.]



Kuva 5: Erilaiset uhkatiedot muunnetaan vektoreiksi upotusmallin avulla jatkoanalyysiä varten.

Kuvassa 5 esitetään, kuinka RAG-arkkitehtuurissa käytettävä upotusmalli (embedding model) muuntaa syötetyn uhkatiedon numeerisiksi vektoreiksi. Upotusmalli käyttää eri uhkatiedon tyyppejä, kuten IP-osoitteita, haittaohjelmien tiivisteitä ja uhkatoimijoiden tunnistetietoja, ja tuottaa yhtenäisen vektorirepresentaation, joka mahdollistaa tietojen yhdistämisen ja analysoinnin.

3.3.1 Mitä on Retrieval-Augmented Generation?

Teknisesti RAG-arkkitehtuuri rakentuu vektorihakuteknologiaan. Käyttäjän esittämä kysymys muunnetaan numeeriseksi vektoriedustukseksi upotusmallin (embedding model) avulla. Tätä kysymysvektoria verrataan tietokannan dokumenttien vektoriesityksiin, ja järjestelmä palauttaa semanttisesti lähimmät tietokatkelmat. [32.]

Tämän työn käytännön toteutuksessa vektorihaku on rakennettu ElasticSearch-alustan avulla. ElasticSearch mahdollistaa tehokkaan vektoripohjaisen haun sekä sen yhdistämisen perinteiseen avainsanahakuun (hybridihaku). Näin voidaan hyödyntää sekä semanttista merkitystä että suoraa terminologista vastaavuutta uhkatiedon haussa.

Koska suurilla kielimalleilla (LLM) on rajallinen konteksti-ikkuna, lähdemateriaali on jaettava pienempiin asiakirjalohkoihin ennen vektorointia ja indeksointia. ElasticSearch mahdollistaa tehokkaan lohkojen tallennuksen ja haun ilman tarvetta ulkoisille vektorikannoille, kuten FAISSille tai Annoylle. [33.]

3.3.2 Kielimallit osana uhkatiedon analyysiä ja RAG-menetelmää

Retrieval-Augmented Generation -menetelmä tarjoaa tehokkaan lähestymistavan kyberuhkatiedon analysointiin. Kielimalli voi toimia analyytikon tukena tuottamalla yhteenvetoja, selityksiä ja vastauksia suurista ja jäsentymättömistä tietomassoista. Tällainen tuki nopeuttaa uhkatiedon käsittelyä ja mahdollistaa laajemman kokonaiskuvan muodostamisen. Mallit, kuten GPT-4, Claude ja avoimen lähdekoodin järjestelmät kuten LLaMA 2 ja Mistral 7B ovat esimerkkejä yleisesti käytetyistä ratkaisuista. [30.]

Kielimallien hyödyntäminen edellyttää kuitenkin uhkatiedon asianmukaista esikäsittelyä ja jäsentelyä. Esimerkiksi laajat uhkaraportit, blogitekstit tai tietokantarakenteet on jaettava pienempiin, hallittaviin asiakirjalohkoihin. Tämä mahdollistaa sen, että haettavat tiedot mahtuvat suurten kielimallien rajoitettuun konteksti-ikkunaan ja että vastausten laatu pysyy korkeana myös pidemmissä analyysiketjuissa. [33.]

Rakenteinen data, kuten JSON-tiedostot, taulukot ja tietokantarakenteet, voidaan syöttää kielimalleille esimerkiksi Markdown-muodossa. Tällöin kielimallit pystyvät käsittelemään myös teknistä ja rakenteellista tietoa tehokkaasti osana vastausprosessia. Näin voidaan hyödyntää yhdistettyä lähestymistapaa, jossa rakenteinen ja rakenteeton tieto analysoidaan saumattomasti.

On kuitenkin tärkeää huomioida, että vaikka RAG-arkkitehtuuri tarjoaa edistyksellisiä mahdollisuuksia tiedon seulontaan ja tiivistämiseen, se ei korvaa asiantuntijan tekemää tulkintaa. Automaattinen järjestelmä kykenee seulomaan olennaiset kohdat suurista tietomassoista, mutta lopullinen päätöksenteko ja analyysin validointi edellyttävät edelleen ihmisen kriittistä arviointikykyä.

Retrieval-Augmented Generation toimii siis ennen kaikkea analyytikon tukena. Sen suurin vahvuus on kyky tiivistää ja yhdistää tietoa laajoista, heterogeenisistä lähteistä, jolloin analyytikon työaikaa voidaan kohdentaa vaativampaan tiedon arviointiin ja tulkintaan. Tekoäly helpottaa tiedon keruuta ja esikäsittelyä, mutta analyysin laatu ja käyttökelpoisuus varmistetaan edelleen asiantuntijatyön avulla.

3.3.3 Aiemmat tutkimukset RAG-menetelmän käytöstä kyberturvallisuudessa

Retrieval-Augmented Generation (RAG) -menetelmää on sovellettu yhä laajemmin kyberturvallisuudessa, erityisesti uhkatiedon käsittelyssä ja analysoinnissa. Pacific Northwest National Laboratory on kehittänyt CyRAG- ja GraphCyRAG-järjestelmät, joissa yhdistyvät RAG-arkkitehtuuri ja semanttiset tietograafit, kuten Neo4j-pohjaiset CVE–CAPEC–ATT&CK-verkostot. Nämä ratkaisut mahdollistavat rakenteisen uhkatiedon automaattisen hakemisen ja kontekstualisoinnin analyysia varten. [34.]

GraphCyRAG:n avulla on osoitettu, että kyberuhkakysymyksiin vastaaminen tehostuu merkittävästi, kun tietoa voidaan yhdistää semanttisesti eri lähteistä [34.] Myös AttackER-tietopohjaan perustuva RAG-järjestelmä on parantanut johdonmukaisuutta ja lähteisiin sidottua vastaustarkkuutta verrattuna pelkkiä kielimalleja hyödyntäviin ratkaisuihin. [35.]

On kuitenkin huomioitava, että RAG-järjestelmien käyttö ei ole riskitöntä. Tietomyrkytys (data poisoning) muodostaa merkittävän uhan: jos ulkoisiin tietovarastoihin päätyy manipuloitua tai virheellistä tietoa, järjestelmien tuottamat analyysit voivat olla harhaanjohtavia. [36; 37.] Tämän vuoksi tiedon lähteiden validointi ja jatkuva laadunvalvonta ovat keskeisiä RAG-pohjaisten ratkaisujen luotettavuuden varmistamisessa.

4 Järjestelmän suunnittelu ja toteutus

Tämä järjestelmä on suunniteltu tukemaan kyberuhkatiedustelua erityisesti tiedon esikäsittelyn, hakumekanismien ja automaattisen vastausten generoinnin avulla. Toteutus pohjautuu luvussa 3 kuvattuun teoreettiseen malliin ja vastaa suoraan työn päätutkimuskysymyksiin. Koko järjestelmän rakenne on suunniteltu tukemaan erityisesti kysymyksiä 2 ja 3, jotka tarkastelevat tiedon esikäsittelyn ja semanttisen haun roolia hybridimenetelmillä.

Luvussa käsitellään järjestelmän arkkitehtuuria, tiedonkeruumenetelmiä, esikäsittelyprosessia, haku- ja generointimekanismeja sekä käyttöliittymää. Lisäksi tarkastellaan toteutuksessa kohdattuja teknisiä haasteita ja niihin kehitettyjä ratkaisuja.

4.1 Arkkitehtuurin yleiskuvaus

Tässä luvussa kuvataan opinnäytetyönä toteutetun uhkatiedonhallintajärjestelmän tekninen rakenne. Järjestelmä perustuu Retrieval-Augmented Generation (RAG) -arkkitehtuuriin, jossa yhdistyvät uhkatiedon keruu, semanttinen haku ja suurten kielimallien tuottama vastausgenerointi. Tavoitteena oli rakentaa kokonaisuus, joka kykenee käsittelemään sekä rakenteellista että rakenteetonta uhkatietoa, tuottamaan siitä yhteenvetoja ja tarjoamaan tietoturva-asiantuntijalle työkalun kysymys-vastaus-tyyppiseen tiedonhakuun.

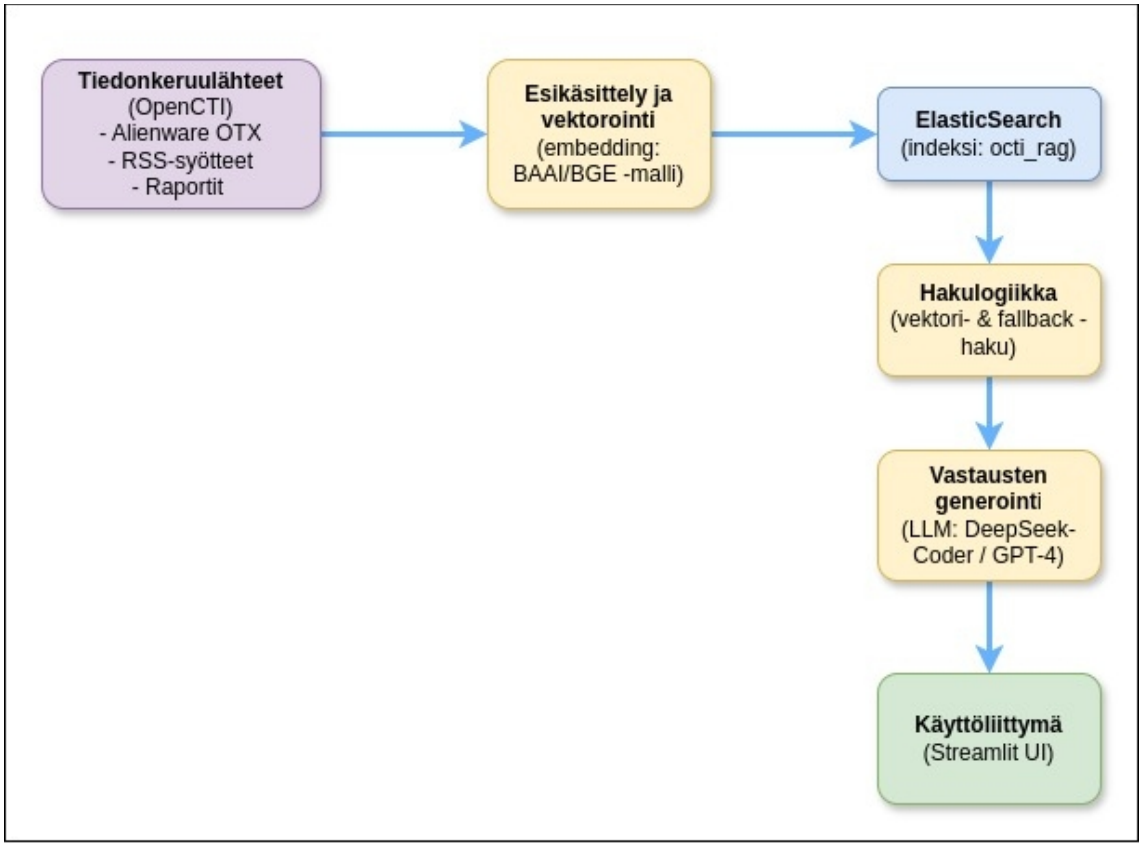
Järjestelmä koostuu neljästä pääkomponentista:

- **OpenCTI-alusta**, jonka avulla uhkatietoa kerätään ja jäsennetään STIX 2.1 -formaatissa
- **Elasticsearch-tietokanta**, johon data tallennetaan vektorimuodossa
- **Kielimallit** (Ollama ja OpenAI), jotka generoivat vastauksia kysymyksiin

- **Streamlit-pohjainen käyttöliittymä**, jonka kautta käyttäjä vuorovaikuttaa järjestelmän kanssa.

Tietovirta alkaa uhkatiedon keräämisestä OpenCTI:n liitännäisten kautta ja etenee datan esikäsittelyn ja embedding-vaiheen kautta Elasticsearchiin. Lopuksi järjestelmä tarjoaa kysely- ja generointikerroksen käyttöliittymässä. Koko järjestelmä on kontitettu Docker-ympäristöön, jossa palvelut käynnistetään *docker-compose.yml*-tiedoston avulla.

Kuvassa 6 on esitetty kokonaisarkkitehtuurin pääkomponentit ja tietovirrat.



Kuva 6: RAG-pohjaisen hakujärjestelmän arkkitehtuuri ja tietovirta uhkatiedon keruusta käyttöliittymään.

4.2 Tiedonkeruu ja lähteet

Järjestelmässä hyödynnettiin useita julkisesti saatavilla olevia lähteitä, joiden kautta saatiin monipuolista ja ajankohtaista uhkatietoa. Tiedonkeruumenetelmiä tarkastellaan erityisesti työn alakysymyksen 1 näkökulmasta, jossa selvitetään, minkälaista uhkatietoa voidaan tehokkaasti kerätä avoimista lähteistä.

Keskeisimmät tiedonlähteet olivat seuraavat:

- **AlienVault OTX (Open Threat Exchange)**, josta kerättiin rakenteellista tietoa indikaattoreista (IoC), kampanjoista, uhkatoimijoista ja TLP-luokitelluista sisällöistä.

- **RSS-syötteet (n = 53)**, jotka sisälsivät puolistrukturoituja uhkatiedotteita eri kansallisilta ja kansainvälisiltä CERT-organisaatioilta, turvallisuusyrityksiltä ja tutkimuslaitoksilta.

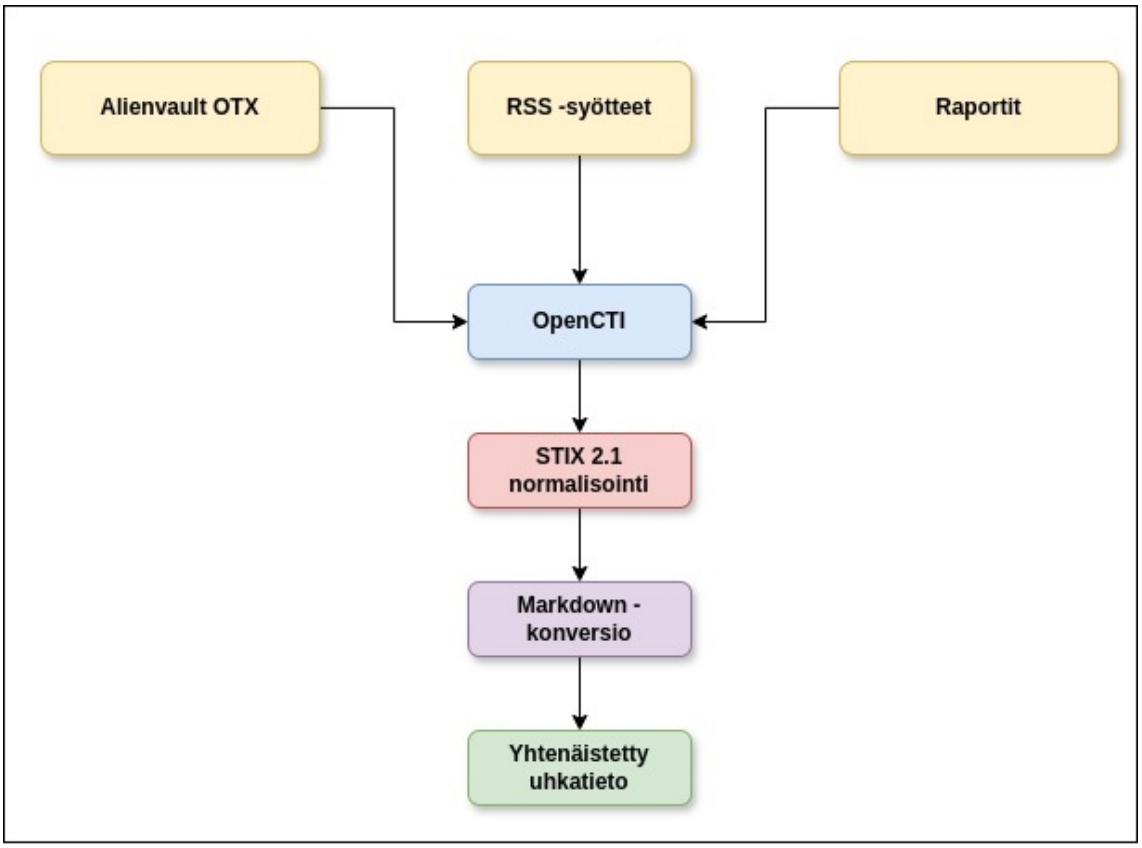
- **Julkaistut uhkatiedon raportit (n > 700)**, jotka konvertoitiin manuaalisesti Markdown-muotoon yhtenäisen jatkokäsittelyn mahdollistamiseksi.

Tiedonkeruuprosessissa hyödynnettiin OpenCTI-alustaa, jonka liitännäisten kautta kerätty aineisto syötettiin järjestelmään ja tallennettiin STIX 2.1 -formaatissa. Tämä mahdollisti uhkatiedon automaattisen luokittelun ja jäsentelyn, kuten haittaohjelmien, hyökkäystekniikoiden ja uhkatoimijoiden tunnistamisen. Kerättyä aineistoa normalisoitiin ja jäsenneltiin, jotta se oli yhteensopivaa järjestelmän muiden osien kanssa.

Tiedonkeruu osoitti, että laajasta avoimesta lähdeaineistosta voidaan kerätä sekä rakenteellista että puolistrukturoitua uhkatietoa, mutta tiedon laatu ja yhtenäisyys vaihtelivat merkittävästi. Tämä korosti tarvetta esikäsittelyyn, jossa epäyhtenäinen data muokattiin analysoitavaksi.

Tiedonkeruuprosessista saatujen tulosten pohjalta voidaan todeta, että RAG-menetelmä voi hyödyntää laajasti eri lähteistä kerättyä tietoa, mutta tiedon tarkkuus ja laatu olivat keskeisiä tekijöitä tulosten relevanssissa.

Kuvassa 7 on esitetty tiedonkeruuprosessi yleisellä tasolla ja sen vaiheistus. Kaikki kerätty aineisto syötettiin OpenCTI-alustaan joko sen vakioliitännäisten tai erillisten skriptien kautta. Tieto normalisoitiin STIX 2.1 -formaatin mukaiseksi, mikä mahdollisti automaattisen uhkatyyppien ja entiteettien luokittelun (esim. haittaohjelmat, hyökkäystekniikat, uhkatoimijat). Tiedonkeruussa saavutettiin hyvä kattavuus, ja järjestelmässä oli käytössä yli 600 000 tietueen kokonaisaineisto eri uhkatietomuodoista.

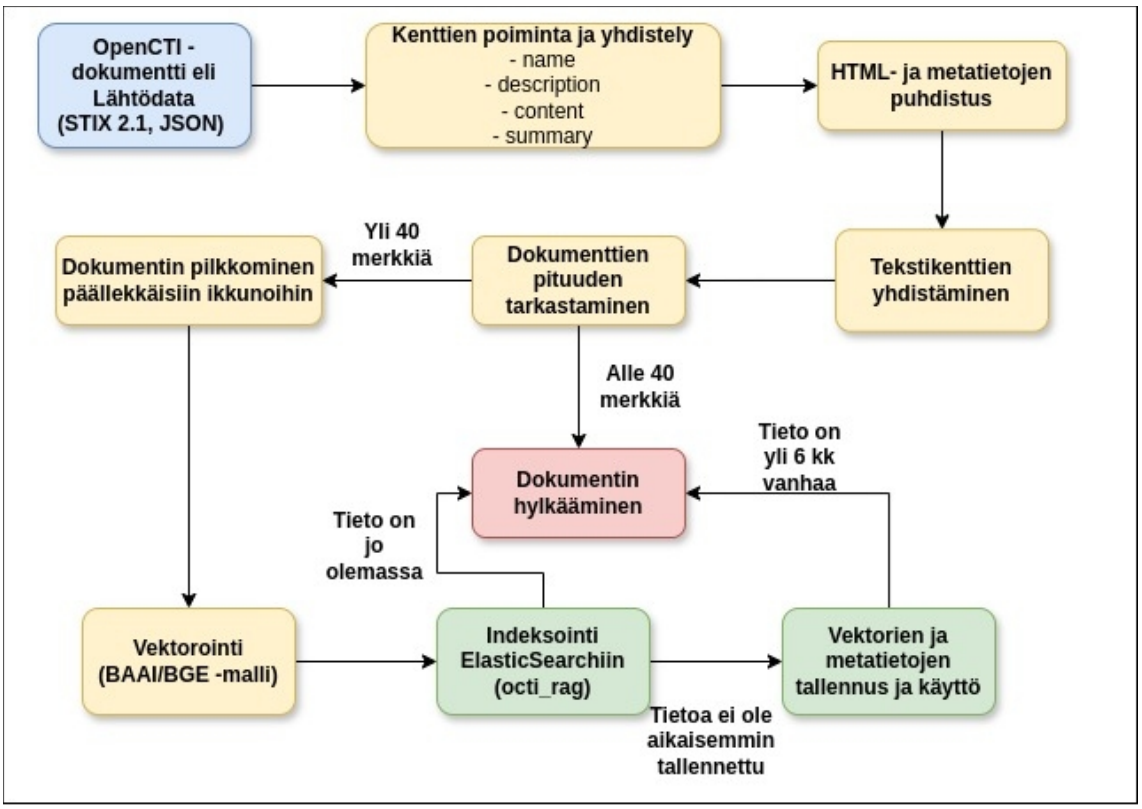


Kuva 7: Tiedonkeruun ja esikäsittelyn yleisprosessi.

4.3 Tiedon esikäsittely ja vektorointi

Tiedon esikäsittely oli järjestelmän toimivuuden kannalta ratkaiseva vaihe, sillä kerätty uhkatieto oli usein epäyhtenäistä ja vaihtelevaa. Esikäsittelyn tavoitteena oli muuntaa raakadata yhtenäiseen ja standardoituun muotoon, joka olisi käyttökelpoinen kielimalleissa ja semanttisessa haussa. Tämä osio liittyy erityisesti tutkimuskysymykseen 2, joka käsittelee tiedon esikäsittelyn vaikutusta kielimallin kykyyn ymmärtää ja käsitellä uhkatietoa.

Järjestelmässä käytettiin OpenCTI:hin tallennettua uhkatietoa, josta poimittiin dokumentteja STIX 2.1 -formaatissa. Esikäsittelyssä poimittiin dokumentin sisältökentät (name, description, content, summary) ja yhdistettiin ne yhtenäiseksi tekstikokonaisuudeksi. Vain tietyt uhkatiedon tyypit hyväksyttiin jatkokäsittelyyn (esim. raportit, uhkatoimijat, haittaohjelmat). Mikäli dokumentti ei täyttänyt ehtoja tai sen sisältö oli liian lyhyt, se jätettiin käsittelemättä.



Kuva 8: Dokumenttien esikäsittely- ja vektorointiprosessi.

Kuva 8 havainnollistaa dokumentin esikäsittelyn ja vektoroinnin prosessia. Kuvassa näkyy, miten dokumentit käsitellään, tarkistetaan pituus ja tallennetaan vektorimuotoon Elasticsearchiin. Tämä prosessi varmistaa, että vain relevantit ja riittävän pitkät dokumentit etenevät embedding-vaiheeseen.

Esimerkkikoodi 1 on peräisin ragtransfer.py-moduulin generate()-funktiosta. Se havainnollistaa, kuinka dokumentista muodostetaan esikäsitelty tekstikenttä ja sen embedding, joka voidaan indeksoida Elasticsearch-järjestelmään:

```
def generate(doc):
    source = doc["_source"]
    entity_type = source.get("entity_type")
    if entity_type not in ALLOWED_TYPES:
        return None
    text = build_text(source)
    if not text or len(text) < 40:
        return None
    embedding = model.encode(text, normalize_embeddings=True).tolist()
    return {
        "_op_type": "index",
        "_index": TARGET_INDEX,
        "_id": doc["_id"],
        "_source": {
            "id_": doc["_id"],
            "report_name": source.get("name", "Unknown"),
            "report_published": source.get("created_at", "Unknown"),
            "source_type": f"OpenCTI {entity_type}",
            "text": text,
            "embedding": embedding,
            "threat_actors": source.get("objectLabel", []),
            "malware": [],
            "ttps": [],
            "url": source.get("external_reference", {}).get("url", "")
        }
    }
```

Esimerkkikoodi 1. Dokumentin esikäsittely ja embeddingin muodostaminen

Tämä koodi esittelee, kuinka dokumentista luodaan esikäsitelty teksti, joka syötetään kielimalliin ja muunnetaan vektoriksi. Vektorointi on tärkeä osa, sillä se luo käsiteltävän aineiston, joka voidaan indeksoida ja hakea Elasticsearch-järjestelmässä.

Markdown-dokumenteista poistettiin HTML- ja metatiedot, jonka jälkeen ne jaettiin kiinteäpituisiin paloihin (chunking). Chunking toteutettiin liukuvalla ikkunalla, jossa osa sisällöstä säilyi päällekkäisenä kontekstin jatkuvuuden säilyttämiseksi. Tämä mahdollisti pitkien tekstien hyödyntämisen kielimallien kontekstirajoissa. Embedding-vaiheessa käytettiin BAAI:n bge-large-en-v1.5-mallia, joka valittiin sen hyvän suorituskyvyn ja kieliteknisen soveltuvuuden perusteella. Malli toimi erityisesti teknisten dokumenttien ja hakutehtävien kanssa erinomaisesti, kuten aiemmassa tutkimuksessa (esim. HuggingFace MTEB) on osoitettu.

Tämän esikäsittelyn ansiosta voitiin vähentää merkittävästi tiedon epäolennaisuutta ja parantaa hakutulosten osuvuutta myöhemmissä järjestelmän vaiheissa. Esimerkiksi, jos dokumentti oli liian lyhyt tai ei täyttänyt ehtoja, se jätettiin käsittelemättä, mikä paransi hakutulosten tarkkuutta ja varmistaa, että vain relevantit tiedot etenevät hakuprosessiin.

4.4 Semanttinen haku ja fallback

Esikäsitelty ja vektorimuotoon muunnettu uhkatieto tallennettiin Elasticsearch-järjestelmään, jossa sitä käytettiin semanttiseen hakuun vektorikentän avulla. Hakujärjestelmä perustui hybridimalliin, jossa yhdistettiin vektoripohjainen haku ja perinteinen avainsanahaku. Indeksi nimeltä *octi_rag* sisälsi seuraavat keskeiset kentät:

- **text** – yhdistetty dokumenttisisältö, jota käytettiin haun kontekstina
- **embedding** – 1024-ulotteinen vektorirepresentaatio tekstistä

- **report_name, source_type, threat_actors, malware, ttps, url** – metatietokenttiä uhkatiedon luokitteluun ja suodatukseen.

Semanttinen haku toteutettiin vertaamalla käyttäjän kysymyksestä muodostettua vektoria indeksin dokumenttien vektoriesityksiin. Vertailussa käytettiin **kosinietäisyyttä**, joka mittaa kahden vektorin välistä kulmaa ja toimii normalisoituna samankaltaisuusmittarina. Mitä pienempi kulma (eli mitä suurempi kosinipistearvo), sitä todennäköisemmin dokumentti vastasi haettua merkitystä. Näin järjestelmä tunnisti semanttisesti vastaavia sisältöjä, vaikka sanamuodot erosivat toisistaan.

Mikäli semanttinen haku tuotti vähemmän kuin kolme osumaa, aktivoitui **fallback-mekanismi**, joka suoritti avainsanapohjaisen haun (match-haku) erityisesti *text*, *report_name* ja *source_type* -kentistä. Fallback-vaihe säilytti järjestelmän toimivuuden myös tilanteissa, joissa vektoripohjainen analyysi epäonnistui esimerkiksi erikoissanaston tai kontekstin puutteiden vuoksi. Tällöin hakujärjestelmä turvautui perinteiseen avainsanahakuun, joka takasi, että hakutuloksia saatiin myös silloin, kun vektoripohjainen haku ei ollut riittävän tarkka.

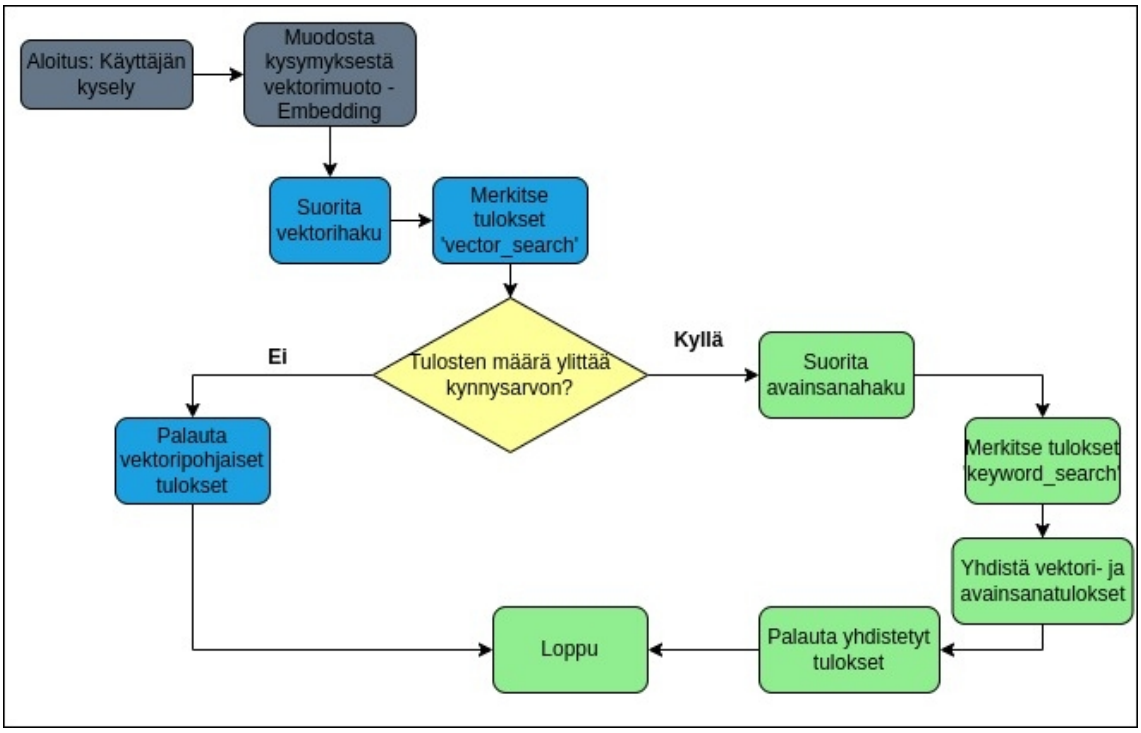
Esimerkkikoodi 2 on otettu suoraan tuotantototeutuksesta (*retrieval.py*) ja havainnollistaa hakuprosessin kulun:

```
def search_with_fallback(query_embedding, query, es_client, index,
top_k=8, threshold=3):
    results = search_index(query_embedding, es_client, index, top_k)
    if len(results) < threshold:
        keyword_results = fallback_keyword_search(query, es_client,
index, top_k)
        for r in keyword_results:
            r["retrieved_by"] = "keyword_search"
        results.extend(keyword_results)
    else:
        for r in results:
            r["retrieved_by"] = "vector_search"
    return results
```

Esimerkkikoodi 2. Hybridihaku: vektorihaku ja fallback-mekanismi

Esimerkkikoodi 2 havainnollistaa, kuinka hakuprosessi etenee. Aluksi suoritetaan vektoripohjainen semanttinen haku, ja jos se ei tuota tarpeeksi osumia (alle kolme), aktivoituu fallback-mekanismi, joka suorittaa avainsanapohjaisen haun. Tällöin hakutulokset palautetaan yhdistettynä molemmista hakumenetelmistä.

Jokaiselle hakutulokselle lisättiin retrieved_by-tunniste, jolla erotettiin semanttisen ja avainsanahaun kautta saadut tulokset. Hakutulosten relevanssia arvioitiin metatietojen perusteella, kuten dokumentin pituuden, lähdeorganisaation tai uhkaluokituksen avulla. Tämä lisäpisteytys paransi lopputulosten osuvuutta ja luotettavuutta.



Kuva 9: Semanttisen ja avainsanapohjaisen hakuprosessin kulku.

Kuva 9 havainnollistaa semanttisen ja avainsanapohjaisen hakuprosessin kulun. Kuvassa on esitetty, kuinka kysely etenee vektoripohjaisesta käsittelystä

fallback-hakuun ja tulosten yhdistämiseen. Tämä prosessi varmistaa, että hakutulokset ovat relevantteja ja kattavia, vaikka vektoripohjainen haku ei tuottaisi tarpeeksi osumia.

4.5 Promptin rakenne ja kielimallien hyödyntäminen

Retrieval-Augmented Generation (RAG) -arkkitehtuurin toimivuus perustuu kehotteen (promptin) tarkkaan muotoiluun. Oikein rakennetut kehotteet antavat kielimallille tarvittavat ohjeet ja kontekstin, mikä takaa täsmälliset ja luotettavat vastaukset. Tämä prosessi on keskeinen erityisesti tutkimuskysymyksessä 3, jossa tarkastellaan, kuinka RAG-menetelmää voidaan hyödyntää kyberuhkatiedon rikastamisessa ja tiivistämisessä.

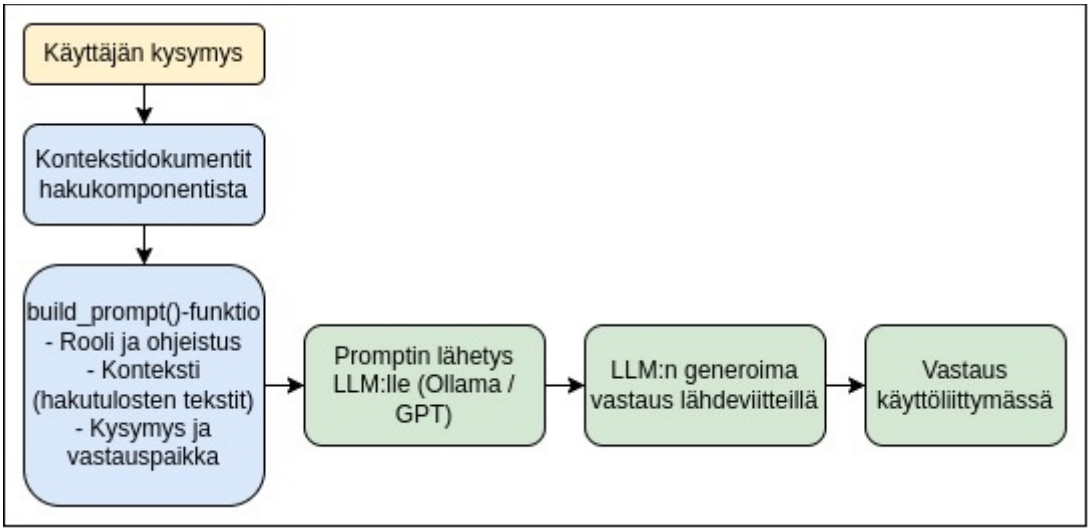
Kehotteen muodostaminen alkaa käyttäjän esittämästä kysymyksestä, joka muunnetaan vektorimuotoon ja lähetetään kielimallille. Prosessi käynnistyy build_prompt()-funktiolla, joka luo kehotteen kolmesta pääkomponentista: rooli ja ohjeistus, konteksti sekä kysymys ja vastauspaikka.

1. **Rooli ja ohjeistus:** Tässä määritellään, mitä roolia kielimalli ottaa. Esimerkiksi, jos kysymys liittyy kyberuhkatiedusteluun, rooli voi olla "Kyberuhkatiedustelun analyytikko". Tämä rooli antaa kielimallille ohjeet siitä, kuinka sen tulee vastata ja mitä lähteitä se voi käyttää.

2. **Konteksti:** Tässä osassa yhdistetään kaikki tarvittavat hakutulokset ja dokumentit, jotka liittyvät käyttäjän kysymykseen. Konteksti luo pohjan, jonka avulla kielimalli voi tuottaa tarkkoja ja asiayhteyteen perustuvia vastauksia.

3. **Kysymys ja vastauspaikka:** Käyttäjän esittämä kysymys lisätään kehotteeseen ja määritellään, mihin kohtaan vastaus tulee kehotteessa (esim. "Vastaus:").

Kun kehotteet on muodostettu, ne lähetetään kielimallille, joka generoi vastauksen ottaen huomioon syötetyn kontekstin. Oikein rakennettujen kehotteiden avulla voidaan minimoida kielimallin virheellisyys ja varmistaa, että vastaukset ovat luotettavia ja selkeästi viitteistetty.



Kuva 10: Kehotteen muodostus ja vastausprosessin eteneminen kielimallilla.

Kehotteen muodostus on tärkeä osa järjestelmän toimintaa, ja sen roolia on havainnollistettu kuvassa 10. esimerkkikoodi 3 näyttää, kuinka build_prompt()-funktio toimii:

```
def build_prompt(context_docs, question):
    header = "You work as Cyber Threat Intelligence Analyst
           assistance. Answer the following question."
    instructions = "Use only the sources provided below and cite them.
           Do not fabricate information."
    context_block = "\n\n".join(doc["text"] for doc in context_docs)
    prompt = f"{header}\n\n{instructions}\n\n{context_block}
           \n\nKysymys: {question}\nVastaus:"
    return prompt
```

Esimerkkikoodi 3. Kehotteen muodostus kielimallia varten build_prompt()-funktion avulla.

Koodissa määritellään ensin rooli ja ohjeistus, sitten kerätään konteksti ja lopuksi lisätään käyttäjän kysymys. Tämä mahdollistaa mallin tuottaman vastauksen, joka on asiayhteyteen perustuva ja vii.

Kehotteen lähettäminen kielimallille johtaa asiayhteyteen perustuvan vastauksen generointiin. Tämä vaihe testaa RAG-menetelmän kyvyn yhdistää eri lähteistä saatu tieto ja tuottaa tarkkoja ja merkityksellisiä vastauksia.

Kielimallien testauksessa käytettiin kahta eri kielimalliperhettä: Ollama-pohjaisia malleja, jotka ajettiin paikallisesti yksityisyyden säilyttämiseksi, ja OpenAI:n GPT-3.5 ja GPT-4, joiden kontekstirajoitukset rajoittivat niiden käyttöä.

Kontekstin tiivistämiseen käytettiin compress_context()-funktiota, joka poimii sisällöllisesti tärkeimmät osat ja säilyttää vastauksen tarkkuuden, vaikka kontekstidokumentit ylittävät kielimallin kontekstirajoitukset.

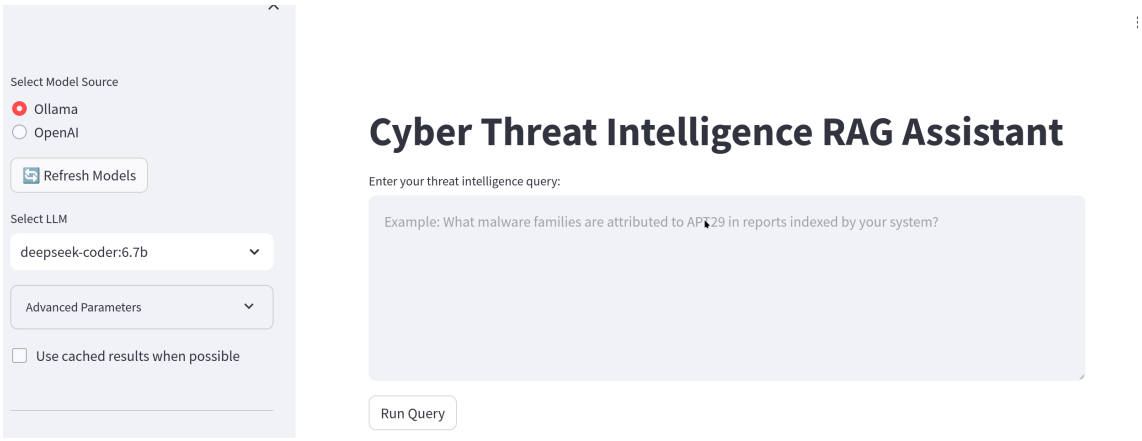
4.6 Käyttöliittymän toteutus

Järjestelmän käyttöliittymä toteutettiin Pythonin **Streamlit**-kirjastolla, jonka etuina olivat nopea prototyyppikehitys, yksinkertainen komponenttien hallinta ja mahdollisuus integroida suoraan Python-pohjaisiin tietojenkäsittelysovelluksiin. Käyttöliittymän päätarkoitus oli tukea järjestelmän testattavuutta ja tarjota vuorovaikutteinen näkymä käyttäjän ja taustalla toimivien komponenttien välille. Käyttöliittymän rooli on erityisesti testausvaiheessa keskeinen, sillä se mahdollistaa käyttäjän ja järjestelmän komponenttien välisten vuorovaikutusten selkeän havainnoinnin.

Käyttöliittymä toimii selainpohjaisesti ja mahdollistaa useita keskeisiä toimintoja, kuten kysymysten syöttö, mallin valinta, parametrien säätö, vastausten tarkastelu, lähteiden tarkastelu ja debug-tiedon seuraaminen. **Kuva 11** esittää käyttöliittymän yleisnäkymän, jossa on esitelty hakukysymys, vastausosio ja järjestel-

män ilmoitukset. Käyttäjä voi esittää vapaamuotoisen kysymyksen, kuten "What malware families are attributed to APT29?" ja järjestelmä palauttaa haetun dokumenttikontekstin perusteella rakennetun vastauksen.

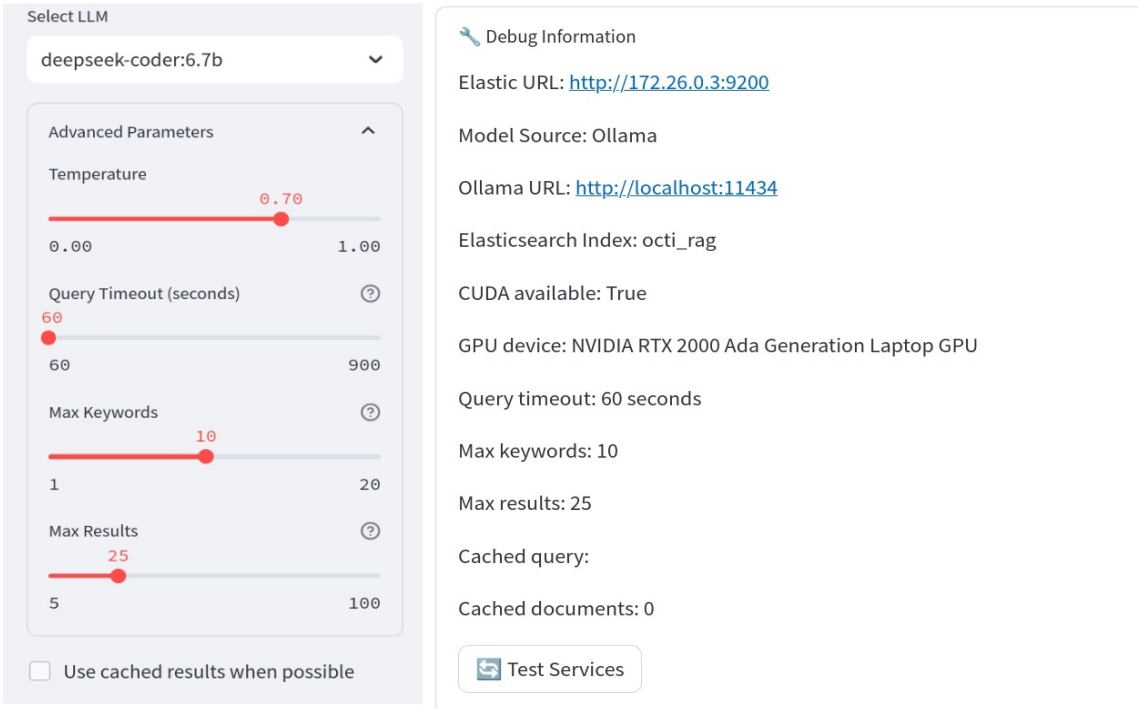
Kuva 12 yhdistää mallin valinnan, säätömahdollisuudet ja debug-tiedot yhdeksi



Kuva 11: Yleisnäkymä kysymyskentästä ja valikosta.

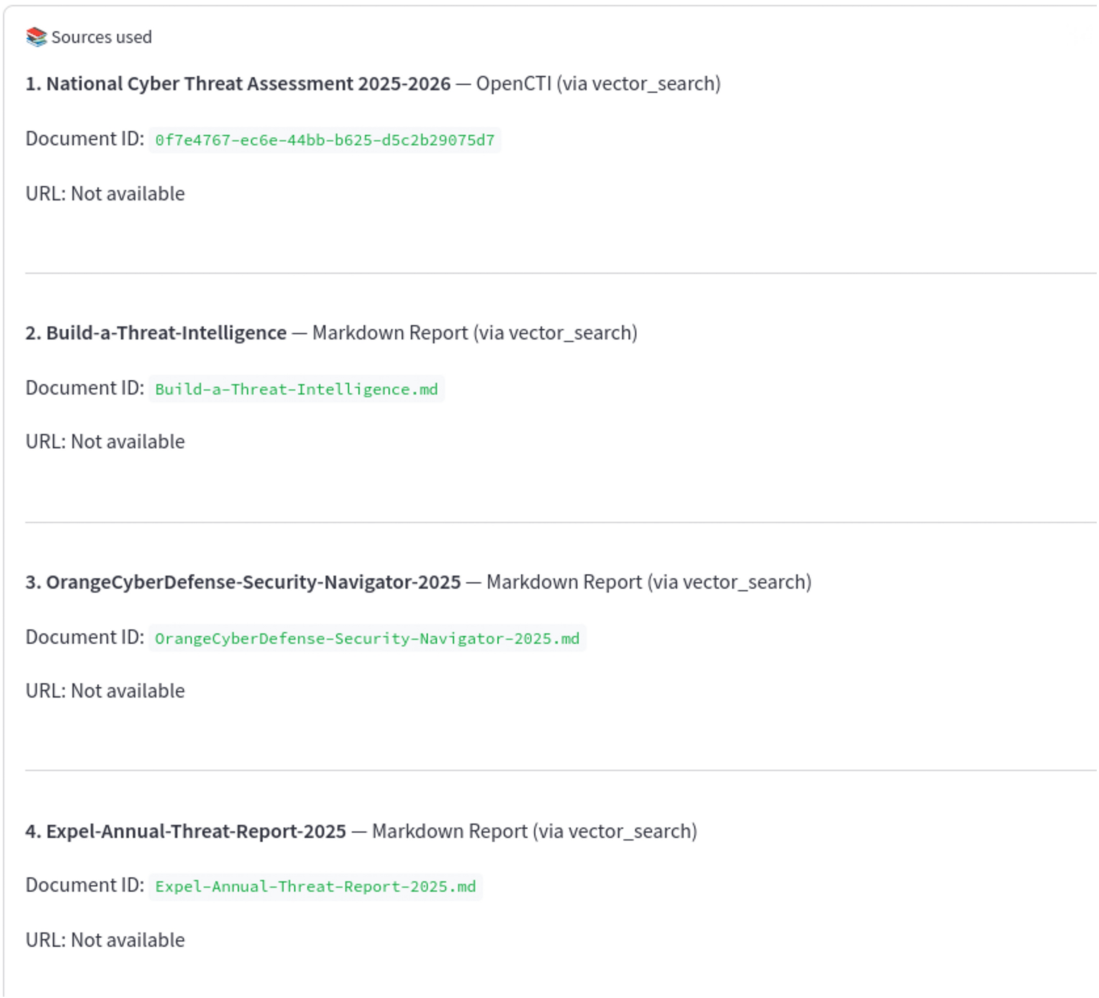
näkymäksi. Vasemmalla puolella käyttöliittymässä käyttäjä voi valita käytettävän kielimallin joko paikallisesti ajetusta Ollama-palvelusta tai OpenAI:n rajapinnasta. Lisäksi voidaan säätää useita parametreja, kuten generoinnin temperature-arvoa, aikarajaa (timeout), fallback-haun avainsanojen määrää (max_keywords) ja palautettavien dokumenttien maksimimäärää (max_results). Tämän avulla järjestelmän käyttäytymistä voidaan hienosäätää vastaamaan erityyppisiä kyselytarpeita.

Oikealla puolella näkyy käyttöliittymän debug-osio, joka paljastaa muun muassa käytössä olevan mallin, Elasticsearch-palvelimen osoitteen, käytössä olevan GPU-laitteiston, cachetut haut ja dokumenttimäärän. Tämä näkymä oli erityisen hyödyllinen järjestelmän testaus- ja virheenselvitysvaiheessa, sillä se mahdollisti nopean tilannearvion ja toistettavat testit.



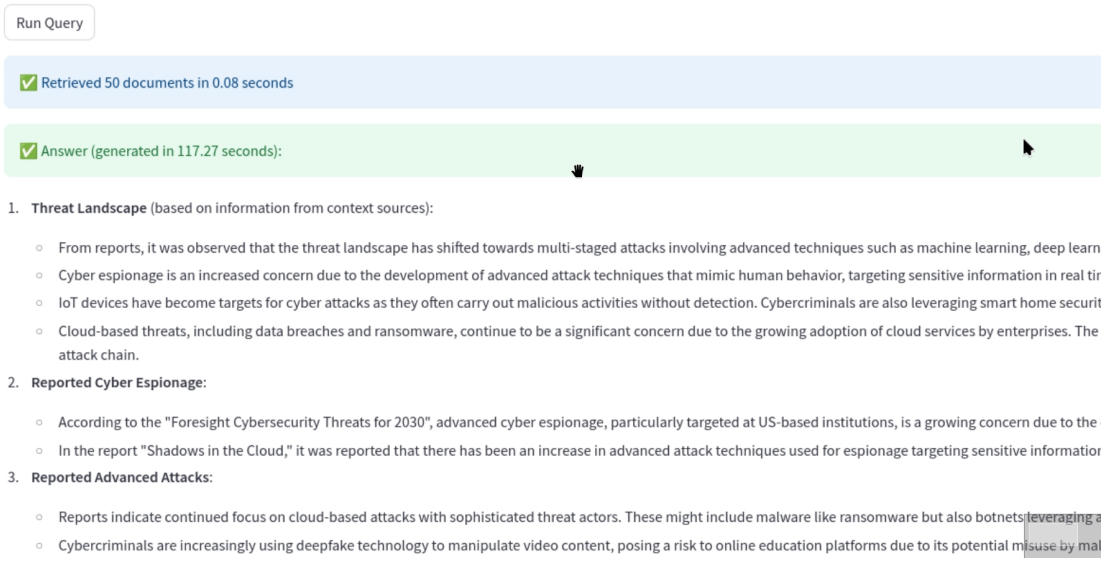
Kuva 12: Mallin valinta, parametrien säätö ja järjestelmätilan debug-näkymä

Hakutulosten laadun ja läpinäkyvyyden parantamiseksi käyttöliittymässä esitetään kaikki vastauksessa käytetyt lähdedokumentit omassa osiossaan. Kuva 13 esittää lähteiden esitystavan, jossa jokainen dokumentti on nimetty, varustettu dokumentti-ID:llä ja merkitty haun perusteella joko semanttiseksi (vector_search) tai avainsanapohjaiseksi (keyword_search). Tällä tavoin käyttäjä voi jäljittää, mihin sisältöön vastaus perustuu ja arvioida lähteiden luotettavuutta.



Kuva 13: Lähdedokumenttien esitystapa

Käyttöliittymässä hyödynnettiin Streamlitin session_state-mekanismia, joka säilytti istunnon aikana aiemmat kysymykset, vastaukset ja valitut asetukset. Tämä mahdollisti aiempien tulosten tarkastelun ilman uudelleenkyselyä ja helpotti vastausten vertailua eri mallien ja parametrien välillä. Kuva 14 esittää valmiin vastauksen, jonka järjestelmä generoi käyttäjän kysymykseen kontekstuaalisen haun perusteella. Malli rakentaa yhteenvedon useista eri lähteistä, ja vastauksen rakenne noudattaa johdonmukaista loogista rakennetta, jossa myös lähteet esitetään eksplisiittisesti.



Kuva 14: RAG:n generoima lopullinen vastaus

Kokonaisuutena käyttöliittymä muodostaa hallitun käyttökerroksen, joka yhdistää järjestelmän eri osat yhdeksi loogiseksi ja käyttäjäystävälliseksi kokonaisuudeksi. Sen visuaalinen selkeys, interaktiiviset elementit ja tekninen läpinäkyvyys tukivat niin kehitystyötä kuin lopullista proof-of-conceptin käyttöönottoa. Käyttöliittymän toteutus ja sen tarjoama vuorovaikutteinen kokemus mahdollistavat tehokkaan testauksen ja virheenkorjauksen, mikä on keskeistä PoC-tason järjestelmän kehityksessä.

4.7 Testaus: kysymyksen muodostaminen ja vastausten arviointi

Tässä osiossa syvennytään järjestelmän testausprosessiin, jossa arvioitiin sen kykyä muodostaa relevantteja vastauksia annettuihin kysymyksiin Retrieval-Augmented Generation (RAG) -arkkitehtuurin avulla. Testauksen keskiössä oli järjestelmän kyky yhdistää käyttäjän kysely, haettu konteksti ja kielimallin generointikyky kyberuhkatiedon (CTI) analyysissä. Tavoitteena oli paitsi arvioida vastausten laatua myös ymmärtää, miten eri parametrit ja järjestelmän komponentit vaikuttavat lopputulokseen.

4.7.1 Testijärjestelyt ja järjestelmän toiminta testauksessa

Testaus suoritettiin käyttäen järjestelmän Streamlit-pohjaista käyttöliittymää (kuvattu luvussa 4.6), joka mahdollisti interaktiivisen kyselyiden suorittamisen ja tulosten tarkastelun. Testiprosessi noudatti järjestelmän ydintoimintalogiikkaa:

- **Kyselyn syöttö:** Käyttäjä esitti luonnollisen kielen kyselyn käyttöliittymän kautta.

- **Semanttinen haku ja fallback:** Järjestelmä muunsi kyselyn vektorimuotoon käyttäen BAAI/bge-large-en-v1.5 -mallia ja suoritti ensisijaisesti semanttisen vektorihakun Elasticsearch-tietokannasta (octi_rag-indeksi). Mikäli vektorihaku ei tuottanut riittävästi tuloksia (oletuksena alle 3), `search_with_fallback`-funktio (Koodi 2, luku 4.4) aktivoi avainsanapohjaisen fallback-haun samoista tai erikseen määritellyistä indekseistä. Hakutuloksiin lisättiin `retrieved_by`-tunniste (vector_search tai keyword_search) erottamaan haun tyyppi.

- **Kontekstin muodostus:** Haetut dokumentit (säädettävä määrä `max_results`-parametrilla) kerättiin kontekstiksi. Pitkien dokumenttien tai suuren dokumenttimäärän tapauksessa `compress_context()`-funktiota (liite 2) käytettiin kontekstin tiivistämiseen kielimallin token-rajan (n. 4000 tokenia testatuilla malleilla) puitteisiin.

- **Kehotteen (Prompt) generointi:** `build_prompt()`-funktio (Koodi 3, luku 4.5 ja liite 2) rakensi kielimallille (esim. testattu DeepSeek-Coder 6.7b tai muut Ollama/OpenAI-mallit) kehotteen, joka sisälsi roolimäärittelyn ("Cyber Threat Intelligence Analyst"), ohjeistuksen käyttää vain annettua kontekstia, itse kontekstidokumentit ja lopuksi käyttäjän alkuperäisen kysymyksen.

- **Vastauksen generointi:** Kielimalli generoi vastauksen perustuen saamaansa kehotteeseen ja kontekstiin. Generointia voitiin ohjata `temperature`-parametrilla, joka vaikutti vastauksen luovuuteen ja deterministisyyteen.

- **Tulosten esittäminen:** Generoitu vastaus ja käytetyt lähdedokumentit esitettiin käyttöliittymässä.

4.7.2 Testitapaus ja tulokset

Yhtenä keskeisenä testitapauksena arvioitiin järjestelmän kykyä vastata laajaan, strategiseen kysymykseen, joka edellyttää tiedon syntetisointia useista lähteistä. Esimerkkikyselynä käytettiin: "What was the Threat Landscape in 2024 like?". Tämäntyyppinen kysymys valittiin, koska se edellyttää tiedon syntetisointia useista lähteistä ja järjestelmän oletettiin toimivan tällaisten kysymysten kanssa paremmin kuin yksittäisten, tarkkarajaisten IoC-tietojen käsittelyssä.

Testiajon tulokset, joita havainnollistetaan myös luvassa 14 (luku 4.6), osoittivat järjestelmän toimintaa seuraavasti:

- **Haku:** Järjestelmä haki onnistuneesti 50 relevantiksi arvioitua dokumenttia Elasticsearchista 0,08 sekunnissa. Hakutapaa (vektori/avainsana) ei tässä esimerkissä eritelty, mutta hybridimekanismi varmisti riittävän määrän kontekstia.

- **Generointi:** Kielimalli (tässä tapauksessa oletettavasti jokin Ollama-malli käytettävissä olevien tietojen perusteella) generoi vastauksen 117,27 sekunnissa.

- **Vastaus:** Tuotettu vastaus kokosi yhteenvedon vuoden 2024 uhkakuvasta perustuen ainoastaan haettuihin lähdedokumentteihin. Vastaus kattoi

useita keskeisiä teemoja, kuten monivaiheiset hyökkäykset, tekoälyn (ML/DL) hyödyntämisen hyökkäyksissä, pilviympäristöjen uhat, lisääntyneen kybervakoilun, IoT-laitteisiin kohdistuvat uhat ja deepfake-teknologian käytön. Vastaus noudatti kehotteen ohjeistusta viitata lähteisiin ja jäsentää tietoa selkeästi.

4.7.3 Analyysi ja laadullinen tulos

Testitulos vahvisti RAG-arkkitehtuurin potentiaalin strategisen tason kyberuhkatiedon analyysissä. Järjestelmä kykeni yhdistämään tietoa useista dokumenteista ja tuottamaan koherentin, lähteistetyn yhteenvedon monimutkaisesta aiheesta. Tarkasteltaessa yksittäisen testitapauksen tulosta kysymykseen "What was the Threat Landscape in 2024 like?" (kuten liitteessä 3 esitetään), voidaan havaita, että järjestelmän generoima vastaus vastaa hyvin suoraan asetettuun kysymykseen. Vastaus jäsentää uhkakuvaa useisiin eri teemoihin, kuten "Threat Landscape (based on information from context sources)", "Reported Cyber Espionage", "Reported Advanced Attacks", ja "Reported IoT Threats", mikä osoittaa järjestelmän kykyä tunnistaa ja luokitella haetusta kontekstista olennaisia osa-alueita. Vastaus myös kokoaa yhteenvedon näistä teemoista loppuun.

Vastauksen muotoilu, käyttäen otsikointia ja luettelopisteitä, parantaa sen luettavuutta ja ymmärrettävyyttä, vastaten samalla kehotteessa asetettuihin ohjeisiin. Vastaus pyrkii myös viittaamaan lähteisiin ("From reports, it was observed that", "According to the 'Foresight Cybersecurity Threats for 2030'", "Reports indicate..."), mikä lisää sen luotettavuutta ja jäljitettävyyttä. Tämä vahvistaa järjestelmän kyvyn syntetisoida tietoa useista dokumenteista ja tuottaa koherentti, lähteistetty yhteenveto.

Samalla testaus kuitenkin osoitti tiettyjä haasteita ja järjestelmän herkkyyttä tietyille tekijöille. Kysymyksen ja kontekstin yhteensopivuuden osalta, vaikka strateginen kysymys toimi hyvin, testit osoittivat, että tarkempien kysymysten (esim.

yksittäiseen IoC-tietoon tai vähemmän dokumentoituun uhkatoimijaan liittyvien) kohdalla relevanssi heikkeni, jos vektorihaku ei löytänyt riittävän tarkkaa kontekstia. Tällöin fallback-mekanismi auttoi, mutta sen tuottama konteksti ei aina ollut semanttisesti yhtä osuvaa.

Vastausten tarkkuuden ja "Hallusinaatioiden" suhteen järjestelmä pyrki vastaamaan vain annettuun kontekstiin perustuen, mikä vähensi virheellisten väittämien (hallusinaatioiden) riskiä. Kuitenkin, jos konteksti oli puutteellista tai ristiriitaista (johtuen datan epäyhtenäisyydestä), kielimallin kyky tuottaa täysin tarkka ja luotettava vastaus heikkeni. Yksittäisten IoC-tietojen kohdalla tämä riski oli suurempi.

Parametrien vaikutus osoittautui tärkeäksi; parametrien, kuten `max_results` ja `temperature`, säätäminen oli keskeistä. Suurempi `max_results` toi laajemman kontekstin, mutta saattoi lisätä kohinaa ja hidastaa generointia. Matalampi `temperature` tuotti täsmällisempiä, mutta joskus toisteisia vastauksia, kun taas korkeampi arvo lisäsi luovuutta, mutta potentiaalisesti myös epätarkkuutta. `query_timeout` ja `max_keywords` olivat tärkeitä fallback-haun hallinnassa.

Tekniset haasteet ilmenivät esimerkiksi pitkien dokumenttien aiheuttamana kontekstirajoitusten ylittymisenä. Tämä oli konkreettinen haaste, johon `compress_context()`-funktio tarjosi toimivan ratkaisun tiivistämällä sisältöä relevanssin kärsimättä merkittävästi. Fallback-mekanismin tarkkuuden vaihteluun puututtiin parantamalla avainsanojen poimintaa (`extract_keywords`) ja hyödyntämällä `retrieved_by`-tunnistetta tulosten analysoinnissa.

Yhteenvetona voidaan todeta, että testaus osoitti järjestelmän pystyvän menestyksekkäästi käsittelemään ja syntetisoimaan kyberuhkatietoa RAG-menetelmällä erityisesti laajoissa, strategisissa kysymyksissä.

Testitapauksen täysi tulos englanniksi litteroituna on esitetty liitteessä 3.

4.8 Teknisiä haasteita ja ratkaisumalleja

Järjestelmän kehitysprosessissa tunnistettiin useita teknisiä haasteita, jotka kohdistuivat ensisijaisesti datan käsittelyyn, suorituskykyyn ja eri komponenttien väliseen yhteensopivuuteen. Näiden haasteiden ratkaisemisessa keskeiseksi lähestymistavaksi muodostui modulaarinen suunnittelu, joka mahdollisti järjestelmän kriittisten osien itsenäisen kehittämisen ja testaamisen sekä tarvittaessa joustavan vaihtoehtoisen toteutuksen. Tämä lähestymistapa oli välttämätön erityisesti epäyhtenäisen uhkatiedon ja kielimallien rajoitusten kanssa työskennellessä.

Taulukossa 1 esitetään keskeiset toteutuksessa kohdatut haasteet sekä niihin sovelletut ratkaisumallit.

Taulukko 1: Teknisiä haasteita ja toteutettuja ratkaisuja.

Haaste	**Ratkaisu**
STIX-rakenteen epätasaisuus.	Normalisointi *generate()* -funktion kautta.
Pitkät dokumentit ylittivät kontekstirajat.	Chunking ja *compress_context().*
Vektorihaun fallback ei tuottanut osumia.	*retrieved_by* -tunniste ja hybridihaku.
Mallien resurssikuorma oli liian korkea.	Parametrien säätö käyttöliittymässä.
Dockerin yhteensopivuusongelmat.	*Setup.sh* automatisoi ympäristön asentamalla riippuvuudet ja käynnistämällä palvelut oikein.

Haasteiden käsittelyssä korostui järjestelmän modulaarinen rakenne: useimmat kriittiset toiminnot oli mahdollista eriyttää ja tarvittaessa korvata tai ohittaa ilman, että koko järjestelmä menetti toimivuuttaan. Tämä mahdollisti nopean testaamisen erilaisissa kehitysympäristöissä.

4.9 Yhteenveto toteutuksesta

Tässä insinöörityössä kehitetty Retrieval-Augmented Generation (RAG) -pohjainen hakujärjestelmä tarjoaa Proof-of-Concept (PoC) -tasoisen ratkaisun kyberuhkatiedon tehokkaampaan käsittelyyn ja analysointiin. Järjestelmän keskeisenä tavoitteena oli osoittaa, kuinka avoimista lähteistä kerättyä uhkatietoa voidaan hyödyntää automaattisesti ja semanttisesti suurten kielimallien avulla vastauksien generoimiseksi. Toteutus pyrki vastaamaan nykyisten uhkatiedonhallinnan haasteisiin, kuten tiedon pirstaleisuuteen ja analyysiprosessin hitauteen, hyödyntämällä modernia tekoälyarkkitehtuuria.

Järjestelmän tekninen rakenne muodostui viidestä toisiinsa integroidusta pääkomponentista: OpenCTI-alustasta tiedon keruuta ja jäsentelyä varten, esikäsittely- ja vektorointimoduulista, joka muunsi kerätyn datan numeeriseen muotoon, Elasticsearch-tietokannasta semanttista ja hybridihakua varten, promptien muodostuksesta ja kielimallien (kuten Ollama ja OpenAI) hyödyntämisestä vastausten generointiin sekä Streamlit-pohjaisesta käyttöliittymästä käyttäjän vuorovaikutukseen. Järjestelmä kontitettiin Docker-ympäristöön siirrettävyyden ja hallittavuuden parantamiseksi. Käytännön toteutuksessa käsiteltiin laajaa, yli 600 000 dokumentin aineistoa, mikä osoitti menetelmän skaalautuvuutta. Toteutusvaiheen tekniset haasteet, kuten datan epäyhtenäisyys ja kielimallien kontekstirajoitukset, ratkaistiin systemaattisilla esikäsittely- ja hakuoptimoinneilla. Kokonaisuus osoitti RAG-menetelmän potentiaalin uhkatiedon rikastamisessa ja analyysin tukemisessa, erityisesti strategisen ja operatiivisen tason kysymyksissä.

5 Johtopäätökset

Tässä tutkimuksessa kehitettiin ja arvioitiin Retrieval-Augmented Generation (RAG) -pohjainen kyberuhkatiedon hakujärjestelmä, jonka tavoitteena oli tutkia, kuinka tehokkaasti tämä menetelmä voi tukea kyberuhkatiedustelun prosesseja automatisoimalla avoimista lähteistä kerätyn uhkatiedon esikäsittelyn, semantti-

sen haun ja analyysin. RAG-arkkitehtuuri yhdistää semanttisen haun ja suurten kielimallien generointikyvyt, mikä mahdollistaa ajankohtaisen tiedon hyödyntämisen tehokkaasti ja kontekstualisoidusti. Tämä arkkitehtuuri tuo monia etuja, mutta myös huomattavia rajoituksia, jotka on otettava huomioon järjestelmän käytön laajentamisessa ja kehittämisessä.

Tutkimuksen aikana havaittiin, että RAG-pohjainen järjestelmä toimi tehokkaasti erityisesti laajojen, kontekstuaalisesti jäsenneltyjen kysymysten osalta, kuten uhkatoimijoiden ja kampanjarakenteiden analysoinnissa. Järjestelmä pystyi tuottamaan asiantuntevia ja oikeaan kontekstiin sidottuja vastauksia silloin, kun analysoitavaa materiaalia oli runsaasti ja se oli esikäsitelty huolellisesti. Tämä paransi analyysin tarkkuutta ja relevanssia, sillä suurten kielimallien avulla oli mahdollista tiivistää ja jäsentää tietoa tehokkaasti. Kuitenkin, kun järjestelmä joutui käsittelemään pieniä ja teknisiä IoC-tietoja (Indicator of Compromise), sen kyky tuottaa tarkkoja ja relevantteja vastauksia heikkeni merkittävästi. Tämä havainto tuo esiin RAG-menetelmän rajoitteet erityisesti teknisten ja yksityiskohtaisten tietojen käsittelyssä.

RAG-mallin suorituskyvyn rajoitukset ovat linjassa aiempien tutkimusten kanssa, jotka ovat nostaneet esiin tämän menetelmän haasteet erityisesti yksityiskohtaisempien ja teknisempien tietojen käsittelyssä. RAG-pohjainen järjestelmä ei yksinään riitä korvaamaan kyberuhkatiedustelun kokonaisprosessia tai sen vaativaa analyyttistä tulkintaa. Sen sijaan se toimii tehokkaana tukikomponenttina, joka voi tukea asiantuntijatyötä erityisesti silloin, kun tiedon esikäsittely, rakenteellistaminen ja kontekstointi on toteutettu huolellisesti. Tämä tarkoittaa, että RAG:n suurin potentiaali ilmenee osana laajempaa tiedonhallinnan arkkitehtuuria, jossa sen rooli on täydentävä ja automatisointia tukeva, ei korvaava. RAG-malli voi toimia tehokkaana välineenä asiantuntijoiden työssä, mutta se ei voi täysin korvata inhimillistä asiantuntemusta ja kriittistä ajattelua, erityisesti monimutkaisemmissa ja teknisesti vaativissa kyberuhkatiedustelun tehtävissä.

Vaikka RAG-pohjainen järjestelmä tuo merkittävää lisäarvoa kyberuhkatiedusteluun, sen täysimittainen hyödyntäminen edellyttää edelleen merkittäviä kehityasteita. Järjestelmän tehokkuus ja luotettavuus ovat suuresti riippuvaisia datan laadusta ja esikäsittelyn tarkkuudesta. Tiedon epäyhtenäisyys ja puutteellinen validointi olivat keskeisiä haasteita, jotka vaikuttivat järjestelmän tuottamien tulosten tarkkuuteen ja relevanssiin. Tämä tarkoittaa, että tiedon esikäsittelyllä ja validoinnilla on erittäin tärkeä rooli järjestelmän luotettavuuden varmistamisessa. RAG-pohjainen järjestelmä toimii parhaiten silloin, kun data on jäsenneltyä ja esikäsitelty huolellisesti, mutta jos data on epäjohdonmukaista tai puutteellista, järjestelmän suorituskyky heikkenee.

Tämä tutkimus on myös tuonut esiin, kuinka tärkeää on yhdistää RAG-menetelmä muiden kyberuhkatiedustelun työkalujen ja lähteiden kanssa. Yksittäisen RAG-mallin käyttö ei riitä täydentämään asiantuntijan työskentelyä, vaan sen on oltava osa laajempaa tiedonhallinnan ekosysteemiä. RAG voi toimia osana modulaarista ja sääntöpohjaista arkkitehtuuria, jossa sen rooli on rikastaa ja jäsentää dataa, mutta asiantuntijan on edelleen valvottava ja täydentää tuloksia. Tämä korostaa hybridityön ja asiantuntijatyön tärkeyttä, erityisesti silloin, kun käsitellään monimutkaisempia kyberuhkia ja teknisiä IoC-tietoja.

Jatkokehityksessä on useita keskeisiä osa-alueita, jotka vaativat huomiota ja parantamista. Ensinnäkin, RAG-mallin kykyä käsitellä pieniä ja teknisiä IoC-tietoja tulisi parantaa. Tällä hetkellä RAG-malli ei riitä tuottamaan tarkkoja ja luotettavia vastauksia, jos data ei ole riittävän jäsenneltyä. Tämän haasteen ratkaisemiseksi voidaan integroida syväoppimismenetelmiä, kuten konvoluutio- ja toistoverkkoja, jotka pystyvät paremmin yhdistämään teknisiä yksityiskohtia laajempaan uhkakontekstiin. Tällöin RAG-malli voisi tuottaa tarkempia ja relevantimpia vastauksia erityisesti teknisesti yksityiskohtaisille kysymyksille.

Toiseksi, tiedon esikäsittelyn ja validoinnin parantaminen on olennaista RAG-järjestelmän luotettavuuden ja tarkkuuden lisäämiseksi. Tämä voitaisiin saavut-

taa kehittämällä kehittyneempiä validointimekanismeja, kuten monivaiheisia prosesseja tiedon tarkistamiseen ja arviointiin. Tällöin voitaisiin vähentää virheellisten tietojen vaikutusta ja varmistaa, että vain luotettavat ja relevantit tiedot pääsevät analysoitavaksi.

Kolmas kehityssuunta liittyy agenttipohjaisten rakenteiden laajentamiseen. Agenttien integroiminen RAG-pohjaiseen järjestelmään voisi parantaa sen autonomisuutta ja reagointikykyä. Agentit voisivat reagoida nopeasti uusiin uhkiin ja tuottaa analyysejä itsenäisesti ilman jatkuvaa asiantuntijan valvontaa. Tämä laajentaisi järjestelmän skaalautuvuutta ja parantaisi sen tehokkuutta suurten tietomassojen käsittelyssä. Agenttipohjaisuus voisi myös tuoda lisää joustavuutta ja mahdollistaa järjestelmän käytön reaaliaikaisessa uhkaseurannassa.

Ajallisen tiedon hallinnan parantaminen on myös keskeinen kehityssuunta. Kyberuhkatiedon ajantasaisuus on kriittistä, sillä erityisesti IoC-tieto vanhenee nopeasti. RAG-järjestelmän tulisi kyetä automaattisesti tunnistamaan ja poistamaan vanhentuneet tiedot, jotta analyysiprosessit pysyisivät ajantasaisina ja luotettavina. Tätä varten voitaisiin esimerkiksi luoda sisäänrakennettu kellomekanismi, joka varmistaa, että jokaisella tiedolla on aikaleima, auttaa arvioimaan tiedon relevanssia. Tekoälyä voidaan hyödyntää ennakoimaan uhkia ajallisen datan perusteella ja optimoimaan järjestelmän vasteaikaa. Vanhentuneiden tietojen poistaminen tai merkitseminen vähemmän tärkeiksi voisi parantaa järjestelmän käyttökelpoisuutta pitkällä aikavälillä.

Lopuksi, RAG-järjestelmän kehittämisessä tulisi tutkia uusien kielimallien hyödyntämistä, joiden kontekstiruutu on laajempi, esimerkiksi yli 128 000 tokenia sekä hyödyntää sitä yhdessä koneoppimisen ja uusien NLP -prosessien kanssa. Tällöin RAG-malli voisi käsitellä laajempia ja monimutkaisempia tietokokonaisuuksia ilman, että tietoa täytyy pilkkoa tai tiivistää liikaa. Tämä voisi parantaa erityisesti monimutkaisempien kyberuhkien käsittelyä, joissa laajempi konteksti on tärkeää.

Tulevaisuudessa RAG-pohjaiset järjestelmät voivat kehittyä entistä tarkemmiksi ja tehokkaammiksi, kun niitä kehitetään ja yhdistetään muiden analyysityökalujen kanssa. RAG:n rooli kyberuhkatiedustelussa voi olla merkittävä, mutta sen täydellinen hyödyntäminen edellyttää jatkuvaa kehitystä ja integrointia muiden tiedonkeruun ja analyysityökalujen kanssa. Näin RAG voi tukea asiantuntijoita entistä tehokkaammin kyberuhkien ennakoimisessa ja torjumisessa, mikä auttaa parantamaan kyberturvallisuutta ja suojaamaan organisaatioita uhkilta.

Lähteet

1 National Institute of Standards and Technology (NIST). Guide to Cyber Threat Information Sharing (NIST SP 800-150). Gaithersburg: NIST; 2016. Saatavilla: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-150.pdf.> Luettu 25.3.2025.

2 European Union Agency for Cybersecurity (ENISA). ENISA Threat Landscape 2023. Heraklion: ENISA; 2023. Saatavilla: <https://www.enisa.europa.eu/publications/enisa-threat-landscape-2023.> Luettu 25.3.2025.

3 National Cyber Security Centre (NCSC UK). Cyber Threat Intelligence in Government: A Guide for Decision Makers & Analysts. London: NCSC; 2020. Saatavilla: <https://hodigital.blog.gov.uk/wp-content/uploads/sites/161/2020/03/Cyber-Threat-Intelligence-A-Guide-For-Decision-Makers-and-Analysts-v2.0.pdf.> Luettu 25.3.2025.

4 Palo Alto Networks. What are the Types of Cyberthreat Intelligence (CTI)?. 2024. Saatavilla: <https://www.paloaltonetworks.com/cyberpedia/types-of-cyberthreat-intelligence.> Luettu 25.3.2025.

5 Joint Chiefs of Staff, Department of Defense. Doctrine for the Armed Forces of the United States. 2.painos. Washington, DC: Independently Published; 2019.

6 National Cyber Security Centre (NCSC UK). Cyber Threat Intelligence in Government: A Guide for Decision Makers & Analysts. London: NCSC;

2020. Saatavilla: <https://hodigital.blog.gov.uk/wp-content/uploads/sites/161/2020/03/Cyber-Threat-Intelligence-A-Guide-For-Decision-Makers-and-Analysts-v2.0.pdf.> Luettu 25.3.2025.

7 Heuer RJ. The Psychology of Intelligence Analysis. Washington, DC: Center for the Study of Intelligence; 1999.

8 Joint Task Force Transformation Initiative. Guide for Conducting Risk Assessments. NIST SP 800-30 Rev. 1. Gaithersburg (MD): NIST; 2012. Saatavilla: <https://doi.org/10.6028/NIST.SP.800-30r1.> Luettu 1.4.2025.

9 Caballero J, Gomez G, Matic S, Sánchez G, Sebastián S, Villacañas A. The Rise of GoodFATR: A Novel Accuracy Comparison Methodology for Indicator Extraction Tools. Future Generation Computer Systems. 2022;144:4–6. Saatavilla: <https://doi.org/10.1016/j.future.2023.02.012.> Luettu 26.3.2025.

10 Coull S, Nichols J. AI and the Five Phases of the Threat Intelligence Lifecycle. Mandiant Blog. 2023. Saatavilla: <https://cloud.google.com/blog/topics/threat-intelligence/ai-five-phases-intelligence-lifecycle/> Luettu 26.3.2025.

11 Palo Alto Networks. What is Cyber Threat Intelligence (CTI)?. 2024. Saatavilla: <https://www.paloaltonetworks.com/cyberpedia/what-is-cyberthreat-intelligence-cti/> Luettu 26.3.2025.

12 Gao P, Zhao L, Li J, Qin Z. Enabling Efficient Cyber Threat Hunting With Cyber Threat Intelligence. arXiv. 2020. Saatavilla: <https://arxiv.org/abs/2010.13637> Luettu 24.3.2025.

13 Wang Z. A Systematic Literature Review on Cyber Threat Hunting. arXiv. 2022. Saatavilla: <https://arxiv.org/abs/2212.05310> Luettu 24.3.2025.

14 MITRE Corporation. MITRE ATT&CK Framework. 2023. Saatavilla: <https://attack.mitre.org/> Luettu 25.3.2025.

15 Shaw AK. Next-Generation Cyber Threat Intelligence Platform [väitöskirja]. ProQuest Dissertations; 2024. Saatavilla: <https://search.proquest.com/openview/0e9eb7a79f239443ca78d333d996bb75/> Luettu 26.3.2025.

16 Filigran. Connectors – OpenCTI documentation. 2024. Saatavilla: <https://docs.opencti.io/5.8.X/deployment/connectors> Luettu 26.3.2025.

17 Cai Z, et al. AI-Driven Threat Detection Systems. Sensors. 2023;23(16):7273. Saatavilla: <https://www.mdpi.com/1424-8220/23/16/7273> Luettu 26.3.2025.

18 OASIS Open. Cyber Threat Intelligence Documentation. 2024. Saatavilla: <https://oasis-open.github.io/cti-documentation> Luettu 26.3.2025.

19 Agarwal R, et al. Artificial Intelligence in Cybersecurity: Enhancing Threat Detection and Mitigation. ResearchGate. 2024. Saatavilla: <https://www.researchgate.net/publication/377245502> Luettu 26.3.2025.

20 Lu W, Xu S, Yi X. Optimizing Active Cyber Defense. Teoksessa: Decision and Game Theory for Security. LNCS 8252. Cham: Springer; 2013. s. 206–225.

21 Bai H, et al. AI-Augmented Threat Intelligence Framework. arXiv. 2024. Saatavilla: <https://arxiv.org/abs/2410.20287> Luettu 26.3.2025.

22 Bountakas P, Fysarakis K, Kyriakakis T. SYNAPSE – An Integrated Cyber Security Risk & Resilience Management Platform. Proceedings of the ACM Symposium on QoS and Security; 2024. Saatavilla: <https://dl.acm.org/doi/10.1145/3664476.3669924> Luettu 26.3.2025.

23 Uptycs Threat Research Team. Machine learning in cybersecurity: Applications and challenges. Uptycs Blog. Saatavilla: <https://www.uptycs.com/blog/threat-research-report-team/machine-learning-in-cybersecurity> Luettu 17.4.2025.

24 LeCun Y, Bengio Y, Hinton G. Deep learning. Nature. 2015;521(7553):436–444. Saatavilla: <https://www.cs.toronto.edu/~hinton/absps/NatureDeepReview.pdf> Luettu 17.4.2025.

25 CrowdStrike. Threat Intelligence: What it is and how to use it. CrowdStrike Cybersecurity 101. Saatavilla: <https://www.crowdstrike.com/en-us/cybersecurity-101/threat-intelligence/> Luettu 17.4.2025.

26 Bommasani R, Hudson DA, Adeli E, Altman R, Arora S, von Arx S, et al. On the Opportunities and Risks of Foundation Models. Center for Research on Foundation Models (CRFM), Stanford University; 2021. Saatavilla: <https://crfm.stanford.edu/report.html> Luettu 17.4.2025.

27 Elastic. What is vector search? Elastic; 2024. Saatavilla: <https://www.elastic.co/what-is/vector-search/> Luettu 17.4.2025.

28 Song X, Salcianu A, Song Y, Dopson D, Zhou D. Fast WordPiece Tokenization. arXiv; 2020. Saatavilla: <https://arxiv.org/pdf/2012.15524> Luettu 17.4.2025.

29 IBM Research. What is retrieval-augmented generation? IBM Research Blog. 2023. Saatavilla: <https://research.ibm.com/blog/retrieval-augmented-generation-RAG> Luettu 26.3.2025.

30 Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. Advances in Neural Information Processing Systems (NeurIPS); 2020. Saatavilla: <https://arxiv.org/abs/2005.11401> Luettu 26.3.2025.

31 Borgeaud S, Mensch A, Hoffmann J, Cai T, Rutherford E, Millican K, et al. Improving language models by retrieving from trillions of tokens. DeepMind Research. 2022. Saatavilla: <https://arxiv.org/abs/2112.04426> Luettu 26.3.2025.

32 NVIDIA. What Is Retrieval-Augmented Generation (RAG)?. NVIDIA Blog. 2024. Saatavilla: <https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/> Luettu 26.3.2025.

33 Rahman MDS, Rabbi SME, Rashid MM. Optimizing Domain-Specific Image Retrieval: A Benchmark of FAISS and Annoy with Fine-Tuned Features. arXiv. 2024. Saatavilla: <https://arxiv.org/abs/2412.01555> Luettu 26.3.2025.

34 Rahman F, Bramer L, Curiel R. Retrieval Augmented Generation for Robust Cyber Defense (CyRAG and GraphCyRAG). Pacific Northwest National Laboratory (PNNL); 2024. Saatavilla: <https://www.pnnl.gov/publications/cyrag> Luettu 26.3.2025.

35 Rajapaksha S, Rani R, Karafili E. A RAG-Based Question-Answering Solution for Cyber-Attack Investigation and Attribution. arXiv. 2024. Saatavilla: <https://arxiv.org/abs/2402.06407> Luettu 26.3.2025.

36 Jagielski M, Carlini N, Lee H, Tramer F, Roberts D, Madry A. Threat of Data Poisoning in Retrieval-Augmented Generation. USENIX Security Symposium; 2023. Saatavilla: <https://www.usenix.org/conference/usenixsecurity23/presentation/jagielsk > Luettu 26.3.2025.

37 Mezzi E, Massacci F, Tuma K. Large Language Models are Unreliable for Cyber Threat Intelligence. arXiv. 2024. Saatavilla: <https://arxiv.org/pdf/2503.23175> Luettu 13.4.2025.

Liite 1 – Järjestelmän konttikonfiguraatio (docker-compose.yml)

```
# Main OpenCTI services
services:
  # OpenCTI Core Platform
  redis:
    image: redis:7.4.2
    restart: always
    volumes:
      - redisdata:/data
    networks:
      - opencti-network
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 3

  elasticsearch:
    image: docker.elastic.co/elasticsearch/elasticsearch:8.17.4
    container_name: elasticsearch
    volumes:
      - /home/elastic_data:/usr/share/elasticsearch/data
    environment:
      - discovery.type=single-node
      - xpack.ml.enabled=false
      - xpack.security.enabled=false
      - thread_pool.search.queue_size=5000
      - logger.org.elasticsearch.discovery="ERROR"
      - ES_JAVA_OPTS=-Xms8g -Xmx16g
    restart: always
    networks:
      - opencti-network
    ulimits:
      memlock:
        soft: -1
        hard: -1
      nofile:
        soft: 65536
        hard: 65536
    healthcheck:
      test: curl -s http://elasticsearch:9200 >/dev/null || exit 1
      interval: 30s
      timeout: 10s
      retries: 50
```

```
  minio:
    image: minio/minio:RELEASE.2024-05-28T17-19-04Z
    volumes:
      - s3data:/data
    ports:
      - "9000:9000"
    environment:
      MINIO_ROOT_USER: ${MINIO_ROOT_USER:-redacted}
      MINIO_ROOT_redacted: ${MINIO_ROOT_redacted:-redacted}
    command: server /data
    restart: always
    networks:
      - opencti-network
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
      interval: 30s
      timeout: 20s
      retries: 3

  rabbitmq:
    image: rabbitmq:4.0-management
    environment:
      - RABBITMQ_DEFAULT_USER=${RABBITMQ_DEFAULT_USER:-redacted}
      - RABBITMQ_DEFAULT_PASS=${RABBITMQ_DEFAULT_PASS:-redacted}
      - RABBITMQ_NODENAME=rabbit01@localhost
    volumes:
      - amqpdata:/var/lib/rabbitmq
    restart: always
    networks:
      - opencti-network
    healthcheck:
      test: rabbitmq-diagnostics -q ping
      interval: 30s
      timeout: 30s
      retries: 3

  opencti:
    image: opencti/platform:6.5.11
    restart: always
    environment:
      - NODE_OPTIONS=--max-old-space-size=8096
      - APP__PORT=8080
      - APP__BASE_URL=${OPENCTI_BASE_URL:-http://localhost:8080}
      - APP__redacted__EMAIL=${OPENCTI_redacted_EMAIL:-redacted@redacted.io}
      - APP__redacted__redacted=${OPENCTI_redacted_redacted:-redacted}
      - APP__redacted__TOKEN=${OPENCTI_redacted_TOKEN:-c0ffee00-1337-4def-b33f-
deadbeef0000}
      - APP__APP_LOGS__LOGS_LEVEL=info
      - REDIS__HOSTNAME=redis
```

```
    - REDIS__PORT=6379
    - ELASTICSEARCH__URL=http://elasticsearch:9200
    - ELASTICSEARCH__NUMBER_OF_REPLICAS=0
    - MINIO__ENDPOINT=minio
    - MINIO__PORT=9000
    - MINIO__USE_SSL=false
    - MINIO__ACCESS_KEY=${MINIO_ROOT_USER:-redacted}
    - MINIO__SECRET_KEY=${MINIO_ROOT_redacted:-redacted}
    - RABBITMQ__HOSTNAME=rabbitmq
    - RABBITMQ__PORT=5672
    - RABBITMQ__PORT_MANAGEMENT=15672
    - RABBITMQ__MANAGEMENT_SSL=false
    - RABBITMQ__USERNAME=${RABBITMQ_DEFAULT_USER:-redacted}
    - RABBITMQ__redacted=${RABBITMQ_DEFAULT_PASS:-redacted}
    - SMTP__HOSTNAME=localhost
    - SMTP__PORT=25
    - SMTP__FROM=redacted@opencti.io
    - PROVIDERS__LOCAL__STRATEGY=LocalStrategy
  ports:
    - "8080:8080"
  depends_on:
    redis:
      condition: service_healthy
    elasticsearch:
      condition: service_healthy
    minio:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
  networks:
    - opencti-network
  healthcheck:
    test: ["CMD", "curl", "-f", "http://localhost:8080/"]
    interval: 30s
    timeout: 30s
    retries: 3
    start_period: 120s

# LLM service - Ollama
ollama:
  image: ollama/ollama:latest
  container_name: ollama
  volumes:
    - /nvme0n1p2/ollama_data:/root/.ollama
  ports:
    - 11434:11434
  environment:
    - OLLAMA_HOST=0.0.0.0
```

```
      - OLLAMA_KEEP_ALIVE=24h
      - OLLAMA_NUM_THREADS=16
    restart: always
    networks:
      - opencti-network

  # MITRE ATT&CK connector
  connector-mitre:
    build:
      context: ./opencti-rag-project/mitre-attack-connector
      dockerfile: Dockerfile
    environment:
      - OPENCTI_URL=http://opencti:8080
      - OPENCTI_TOKEN=${OPENCTI_redacted_TOKEN:-REDACTED}
      - CONNECTOR_ID=${CONNECTOR_MITRE_ID:-mitre-connector}
      - CONNECTOR_TYPE=EXTERNAL_IMPORT
      - CONNECTOR_NAME=MITRE ATT&CK
      - CONNECTOR_SCOPE=identity,attack-pattern,course-of-action,intrusion-
set,malware,tool,x-mitre-matrix,x-mitre-tactic
      - CONNECTOR_CONFIDENCE_LEVEL=75
      - CONNECTOR_UPDATE_EXISTING_DATA=true
      - CONNECTOR_RUN_AND_TERMINATE=false
      - CONNECTOR_LOG_LEVEL=info
      - MITRE_INTERVAL=7
    restart: always
    networks:
      - opencti-network
    depends_on:
      opencti:
        condition: service_started

  # AlienVault OTX connector
  connector-alienvault:
    build:
      context: ./opencti-rag-project/alienvault-otx-connector
      dockerfile: Dockerfile
    environment:
      - OPENCTI_URL=http://opencti:8080
      - OPENCTI_TOKEN=${OPENCTI_redacted_TOKEN:-REDACTED}
      - CONNECTOR_ID=${CONNECTOR_ALIENVAULT_ID:-alienvault-connector}
      - CONNECTOR_TYPE=EXTERNAL_IMPORT
      - CONNECTOR_NAME=AlienVault
      - CONNECTOR_SCOPE=alienvault
      - CONNECTOR_CONFIDENCE_LEVEL=75
      - CONNECTOR_UPDATE_EXISTING_DATA=true
      - CONNECTOR_LOG_LEVEL=info
      - ALIENVAULT_BASE_URL=https://otx.alienvault.com
      - ALIENVAULT_API_KEY=REDACTED
      - ALIENVAULT_TLP=${ALIENVAULT_TLP:-white}
```

```
      - ALIENVAULT_CREATE_OBSERVABLES=true
      - ALIENVAULT_CREATE_INDICATORS=true
      - ALIENVAULT_PULSE_START_TIMESTAMP=2023-01-01T00:00:00
      - ALIENVAULT_REPORT_TYPE=threat-report
      - ALIENVAULT_REPORT_STATUS=New
      - ALIENVAULT_GUESS_MALWARE=false
      - ALIENVAULT_GUESS_CVE=false
      - ALIENVAULT_EXCLUDED_PULSE_INDICATOR_TYPES=FileHash-MD5,FileHash-SHA1
      - ALIENVAULT_ENABLE_RELATIONSHIPS=true
      - ALIENVAULT_ENABLE_ATTACK_PATTERNS_INDICATES=true
      - ALIENVAULT_INTERVAL_SEC=1800
    restart: always
    networks:
      - opencti-network
    depends_on:
      opencti:
        condition: service_started

  # Worker
  worker:
    image: opencti/worker:6.5.11
    environment:
      - OPENCTI_URL=http://opencti:8080
      - OPENCTI_TOKEN=${OPENCTI_redacted_TOKEN:REDACTED}
      - WORKER_LOG_LEVEL=info
    depends_on:
      opencti:
        condition: service_started
    deploy:
      mode: replicated
      replicas: 3
    restart: always
    networks:
      - opencti-network

volumes:
  redisdata:
  esdata:
  s3data:
  amqpdata:
  ollama_data:

networks:
  opencti-network:
    driver: bridge
```

Liite 2 – Kehotteen muodostus ja haku (prompt_template.py ja retrieval.py)

Tämä liite sisältää kaksi keskeistä Python-moduulia, jotka liittyvät järjestelmän generointilogiikkaan ja hakutoimintoon. *prompt_template.py* vastaa kyselykohtaisen kehotteen muodostamisesta, ja *retrieval.py* sisältää hybridihakua toteuttavan search_with_fallback-funktion. Koodit ovat osa tuotantoversiota ja dokumentoitu työn tekstiosassa luvussa 4.

prompt_template.py

```
def count_tokens(text, approx_chars_per_token=4):
    """Roughly estimate token count using character count."""
    return len(text) // approx_chars_per_token

def compress_context(context, max_tokens=4000, approx_chars_per_token=4):
    """Compress context to fit within token limit."""
    estimated_tokens = count_tokens(context, approx_chars_per_token)

    if estimated_tokens <= max_tokens:
        return context

    # Split into documents
    documents = context.split("\n\n")

    # Calculate target size per document
    target_size = (max_tokens * approx_chars_per_token) // len(documents)

    # Truncate each document
    compressed_docs = []
    for doc in documents:
        doc_parts = doc.split("**Content Preview:**")
        if len(doc_parts) == 2:
            header, content = doc_parts

            # Keep header intact, truncate content
            content_target_size = target_size - len(header)
            if content_target_size > 0 and len(content) > content_target_size:
                truncated_content = content[:content_target_size] + "..."
                compressed_docs.append(header + "**Content Preview:**" +
truncated_content)
            else:
                compressed_docs.append(doc)
        else:
```

```
            compressed_docs.append(doc)

    return "\n\n".join(compressed_docs)

def build_prompt(context, question, max_tokens=4000):
    """Build a prompt with context compression if needed."""
    # Compress context if needed
    compressed_context = compress_context(context, max_tokens)

    return f"""## Instructions

You are a specialized Cyber Threat Intelligence (CTI) analyst with deep
expertise in analyzing threat actors, malware, and attack patterns. Respond to
the question using ONLY the information in the provided context documents.

## Context Documents

{compressed_context}

## Question

{question}

## Response Guidelines

1. Answer ONLY based on the information in the context documents.
2. If the context doesn't contain relevant information, state "No relevant
information found in the indexed sources."
3. Cite specific reports by name when referencing information (e.g., "According
to the [Report Name]...")
4. Organize your answer with headings and bullet points when appropriate.
5. For threat actors, include their associated malware, TTPs, and targets when
mentioned.
6. Do not make assumptions beyond what is explicitly stated in the documents.
7. Focus primarily on technical details and evidence rather than general
speculation.
"""
```

Retrieval.py

```
import os

import logging
import socket
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()
```

```
# Helper function to check if a hostname can be resolved
def can_resolve_hostname(hostname):
    try:
        socket.gethostbyname(hostname)
        return True
    except socket.gaierror:
        return False

# Ensure URLs use resolvable hosts
def get_resolvable_url(url):
    if "://" in url:
        protocol, rest = url.split("://", 1)
        if "/" in rest:
            host, path = rest.split("/", 1)
            path = "/" + path
        else:
            host = rest
            path = ""

        # If the hostname can't be resolved, fallback to localhost
        hostname = host.split(":")[0]
        if not can_resolve_hostname(hostname) and hostname != "localhost":
            port = host.split(":")[-1] if ":" in host else "9200"
            return f"{protocol}://localhost:{port}{path}"
    return url

# Fix PyTorch path error
import torch
if hasattr(torch, 'classes'):

# Verify CUDA availability - force GPU usage
cuda_available = torch.cuda.is_available()
cuda_device = torch.cuda.get_device_name(0) if cuda_available else "N/A"

from elasticsearch import Elasticsearch
from sentence_transformers import SentenceTransformer
import numpy as np
from typing import List, Dict
import re
import time

# Logging
logger = logging.getLogger(__name__)
logging.basicConfig(level=logging.INFO)

# Get the URL from app.py if available, else use environment variable with
fallbacks
elastic_url = os.environ.get("ELASTIC_URL", os.getenv("ELASTIC_URL",
"http://localhost:9200"))
elastic_url = get_resolvable_url(elastic_url)
```

```
# Try multiple Elasticsearch URLs if needed
ES_URLS = [
    elastic_url,
    "http://elasticsearch:9200", # Docker network address (if running in Docker)
    "http://localhost:9200", # Local address
]

es_index = os.getenv("ELASTIC_INDEX", "octi_rag")
ollama_url = os.environ.get("OLLAMA_URL", os.getenv("OLLAMA_URL",
"http://localhost:11434"))
ollama_url = get_resolvable_url(ollama_url)

# Try to connect to each Elasticsearch URL
es = None
connected_url = None

for url in ES_URLS:
    logger.info(f"Trying to connect to Elasticsearch at {url}")
    try:
        es_test = Elasticsearch(url, request_timeout=5, retry_on_timeout=True)
        if es_test.ping():
            logger.info(f"Successfully connected to Elasticsearch at {url}")
            es = es_test
            connected_url = url
            break
        else:
            logger.warning(f"Could ping Elasticsearch at {url} but got no
response")
    except Exception as e:
        logger.warning(f"Failed to connect to Elasticsearch at {url}: {e}")

# If we couldn't connect to any URL, create a dummy client
if es is None:
    logger.error("Could not connect to Elasticsearch at any URL. Using dummy
client.")
    logger.error("This will affect search functionality. Check your
Elasticsearch configuration.")
    es = Elasticsearch(ES_URLS[0]) # Create a client anyway for code to work
else:
    logger.info(f"Using Elasticsearch at {connected_url} with index {es_index}")

logger.info(f"Ollama URL: {ollama_url}")
logger.info(f"CUDA available: {cuda_available}, Device: {cuda_device}")

# Define fallback indices for demo purposes
FALLBACK_INDICES = [
    "opencti_stix_domain_objects-000001",
    "opencti_stix_core_relationships-000001",
    "opencti_history-000001"
]
```

```
# Create a fallback document for when nothing is found
FALLBACK_DOC = {
    "report_name": "Fallback: No Results",
    "text": "No relevant documents were retrieved. Try refining your query or
using different keywords.",
    "source_type": "system",
    "threat_actors": [],
    "ttps": [],
    "malware": [],
    "report_published": "N/A",
    "url": "N/A"
}

# Example documents for when Elasticsearch is unavailable
EXAMPLE_DOCS = [
    {
        "report_name": "APT29 (Cozy Bear) Threat Profile",
        "text": """APT29, also known as Cozy Bear, is a sophisticated threat
actor believed to be affiliated with the Russian government. This group has been
active since at least 2008 and has targeted governments, diplomatic entities,
think tanks, and healthcare organizations worldwide.

APT29 is known for their sophisticated tactics and focus on stealth. They employ
a diverse set of malware families including MiniDuke, CosmicDuke, HAMMERTOSS,
and WellMess. In recent operations, they have used StellarParticle, GoldMax, and
Cobalt Strike.

The group is particularly known for their role in the SolarWinds supply chain
attack of 2020, where they compromised the software build system to distribute
malicious updates to thousands of organizations.

APT29 typically focuses on intelligence gathering and data theft rather than
destructive attacks.""",
        "source_type": "EXAMPLE Data (Elasticsearch Unavailable)",
        "threat_actors": ["APT29", "Cozy Bear"],
        "malware": ["MiniDuke", "CosmicDuke", "HAMMERTOSS", "WellMess",
"GoldMax", "Cobalt Strike"],
        "ttps": ["Supply Chain Attacks", "Spear Phishing", "Credential Theft"],
        "report_published": "2023-01-01",
        "url": "https://example.com/apt29-profile"
    },
    {
        "report_name": "APT29 Recent Campaigns",
        "text": """Recent campaigns by APT29 (Cozy Bear) have shown an evolution
in their tactics and tools. In 2020, they were behind the SolarWinds supply
chain attack, using a backdoor called SUNBURST (also known as Solorigate). This
allowed them to gain access to numerous organizations including US government
agencies.

In 2021-2022, they were observed using WellMess malware against diplomatic
targets and COVID-19 vaccine research organizations. They've also increasingly
adopted Cobalt Strike beacons for command and control operations.
```

```
Their phishing campaigns have grown more sophisticated, using legitimate cloud
services for command and control to evade detection. They've been observed
exploiting vulnerabilities in popular VPN services to gain initial access.

APT29 continues to focus on espionage activities, with a particular emphasis on
policy documents, diplomatic communications, and strategic planning
information.""",
        "source_type": "EXAMPLE Data (Elasticsearch Unavailable)",
        "threat_actors": ["APT29", "Cozy Bear", "UNC2452"],
        "malware": ["SUNBURST", "Solorigate", "WellMess", "Cobalt Strike",
"Sibot"],
        "ttps": ["Supply Chain Attacks", "VPN Exploitation", "Cloud-based C2"],
        "report_published": "2022-06-15",
        "url": "https://example.com/apt29-campaigns"
    }
]

# Embedding model - explicitly set device to cuda if available
logger.info("Loading BGE embedding model...")
device = "cuda" if cuda_available else "cpu"
embed_model = SentenceTransformer("BAAI/bge-large-en-v1.5", device=device)
logger.info(f"Loaded embedding model: BAAI/bge-large-en-v1.5 on device:
{device}")

def embed_query(text: str) -> List[float]:
    try:
        return embed_model.encode(text, normalize_embeddings=True).tolist()
    except Exception as e:
        logger.error(f"Embedding failed: {e}")
        return []

def retrieve_documents(query: str, top_k: int = 10, max_keywords: int = 10) ->
List[Dict]:
    # Log the retrieval request with query details
    logger.info(f"Retrieving up to {top_k} documents using query:
'{query[:80]}'...")

    # If Elasticsearch is not available, return example documents
    if es is None or connected_url is None:
        logger.warning("Elasticsearch is unavailable. Using example documents.")
        # Filter example docs based on keyword match
        filtered_docs = []

        # Extract keywords from query
        keywords = extract_keywords(query, max_keywords)
        query_lower = query.lower()

        # Score each example document based on keyword overlap
        for doc in EXAMPLE_DOCS:
            score = 0
            doc_text = doc['text'].lower()
            for keyword in keywords:
```

```
                keyword_lower = keyword.lower()
                if keyword_lower in doc_text:
                    score += 1
                # Check threat actors, malware, ttps
                for field in ['threat_actors', 'malware', 'ttps']:
                    for item in doc.get(field, []):
                        if keyword_lower in item.lower():
                            score += 2 # Higher weight for metadata matches

            if score > 0 or any(term.lower() in doc_text for term in
query_lower.split()):
                filtered_docs.append(doc)

        # Return filtered docs or the example docs if no matches
        return filtered_docs if filtered_docs else EXAMPLE_DOCS[:1]

    docs = {}
    query_vector = embed_query(query)

    # Track if we attempted the primary index
    primary_index_attempted = False

    # 1. Try using octi_rag index first
    logger.info(f"Checking if '{es_index}' exists and has documents...")
    try:
        if es.indices.exists(index=es_index.lower()):
            primary_index_attempted = True
            count = es.count(index=es_index.lower()).get('count', 0)
            logger.info(f"Index '{es_index}' exists with {count} documents")

            if count > 0:
                # 1. Vector search (if embeddings were successful)
                if query_vector:
                    vector_body = {
                        "size": top_k,
                        "query": {
                            "script_score": {
                                "query": {"match_all": {}},
                                "script": {
                                    "source":
"cosineSimilarity(params.query_vector, 'embedding') + 1.0",
                                    "params": {"query_vector": query_vector}
                                }
                            }
                        }
                    }

                    try:
                        vector_res = es.search(index=es_index.lower(),
body=vector_body)
                        for hit in vector_res.get("hits", {}).get("hits", []):
```

```
                            hit["_source"]["retrieved_by"] = "vector_search"
                            docs[hit["_id"]] = hit["_source"]
                        logger.info(f"Vector search retrieved {len(docs)}
documents.")
                    except Exception as e:
                        logger.warning(f"Vector search failed: {e}")

                # 2. Keyword search on main index (always try this as fallback)
                if len(docs) < top_k:
                    keyword_body = {
                        "size": top_k,
                        "query": {
                            "multi_match": {
                                "query": query,
                                "fields": ["text^2", "report_name",
"source_type", "threat_actors", "malware", "ttps"],
                                "fuzziness": "AUTO" # Add fuzziness for better
matching
                            }
                        }
                    }

                    try:
                        keyword_res = es.search(index=es_index.lower(),
body=keyword_body)
                        keyword_hits = 0
                        for hit in keyword_res.get("hits", {}).get("hits", []):
                            if hit["_id"] not in docs:
                                hit["_source"]["retrieved_by"] =
"keyword_search"
                                docs[hit["_id"]] = hit["_source"]
                                keyword_hits += 1
                        logger.info(f"Keyword search added {keyword_hits} new
documents.")
                    except Exception as e:
                        logger.error(f"Keyword search failed: {e}")
            else:
                logger.warning(f"Index '{es_index}' exists but has 0 documents.
Will use fallbacks.")
        else:
            logger.warning(f"Index '{es_index}' does not exist. Will use
fallbacks.")
    except Exception as e:
        logger.error(f"Error checking primary index: {e}")

    # If no documents found or primary index not even attempted, use OpenCTI
indices as fallback for demo
    if not docs:
        logger.warning("No documents found in main index. Using OpenCTI indices
as fallback.")

        for index in FALLBACK_INDICES:
```

```
            try:
                if not es.indices.exists(index=index.lower()):
                    logger.warning(f"Fallback index '{index}' does not exist.")
                    continue

                # Extract entities and keywords from query for better matching
                keywords = extract_keywords(query, max_keywords)
                logger.info(f"Extracted keywords: {keywords}")

                # Build a flexible match query for OpenCTI indices
                query_terms = []
                for keyword in keywords:
                    if len(keyword) > 3: # Skip very short terms
                        query_terms.append(keyword)

                if not query_terms:
                    query_terms = [query] # Use full query if no keywords
extracted

                keyword_body = {
                    "size": max(5, top_k // len(FALLBACK_INDICES)),
                    "query": {
                        "bool": {
                            "should": [
                                {"multi_match": {
                                    "query": term,
                                    "fields": ["name^3", "description^2",
"entity_type", "*_name", "content", "*"]
                                }} for term in query_terms
                            ],
                            "minimum_should_match": 1
                        }
                    }
                }

                keyword_res = es.search(index=index.lower(), body=keyword_body)
                hits = keyword_res.get("hits", {}).get("hits", [])
                logger.info(f"Fallback search on '{index}' found {len(hits)}
documents")

                for hit in hits:
                    doc_id = hit["_id"]
                    source = hit["_source"]

                    # Convert OpenCTI objects to our expected format
                    if doc_id not in docs:
                        formatted_doc = format_opencti_doc(source, index)
                        formatted_doc["retrieved_by"] = "fallback_search"
                        docs[doc_id] = formatted_doc

            except Exception as e:
```

```
                logger.error(f"Error searching fallback index '{index}': {e}")

    # If still no documents found, return example or fallback document
    if not docs:
        logger.warning("No documents found from any search method. Using
example/fallback documents.")
        # Return example documents if "APT29" or "malware" in query
        if "apt29" in query.lower() or "malware" in query.lower():
            for doc in EXAMPLE_DOCS:
                doc["retrieved_by"] = "example_data"
            return EXAMPLE_DOCS
        else:
            FALLBACK_DOC["retrieved_by"] = "fallback_data"
            return [FALLBACK_DOC]

    return list(docs.values())

def extract_keywords(query: str, max_keywords: int = 40) -> List[str]:
    """Extract key entities and terms from the query, limited by max_keywords
parameter"""
    # Extract potential threat actor names (APT followed by number)
    apt_pattern = re.compile(r'\b(APT\d+|[A-Z][a-z]+ ?[A-Z][a-z]+|[A-Z][A-Z0-
9]+)\b')
    apts = apt_pattern.findall(query)

    # Extract potential malware family names (typically CamelCase or ALL_CAPS
with underscores)
    malware_pattern = re.compile(r'\b([A-Z][a-z]+([A-Z][a-z]+)+|[A-Z][A-Z0-9_]+)
\b')
    malware = malware_pattern.findall(query)
    malware = [m[0] for m in malware] #Extract just the first group

    # Extract CVEs
    cve_pattern = re.compile(r'\b(CVE-\d{4}-\d{4,})\b')
    cves = cve_pattern.findall(query)

    # Add other important cyber terms
    cyber_terms = ['malware', 'ransomware', 'backdoor', 'trojan', 'spyware',
'wiper',
                   'phishing', 'infrastructure', 'command', 'control', 'C2',
'attribution']

    # Extract any of these terms in the query
    term_matches = [term for term in cyber_terms if term.lower() in
query.lower()]

    # Combine all extracted terms
    all_terms = apts + malware + cves + term_matches

    # Add tokenized query terms
    query_terms = [q for q in query.split() if len(q) > 3 and q.lower() not in [
```

```
        'what', 'when', 'where', 'which', 'how', 'are', 'the', 'that', 'this',
'these', 'those',
        'your', 'about', 'know', 'information', 'according', 'resources',
'their', 'they',
        'from', 'have', 'with', 'been', 'would', 'could', 'should', 'there',
'were', 'used'
    ]]

    # Combine and limit to max_keywords
    combined_terms = list(set(all_terms + query_terms))

    # Prioritize specific entities (APTs, malware, CVEs) over general terms
    priority_terms = apts + malware + cves
    remaining_slots = max_keywords - len(priority_terms)

    # If we have more priority terms than max_keywords, truncate the priority
terms
    if remaining_slots < 0:
        logger.warning(f"Too many priority terms ({len(priority_terms)}),
truncating to {max_keywords}")
        return list(set(priority_terms))[:max_keywords]

    # Otherwise, fill remaining slots with the other terms
    other_terms = [t for t in combined_terms if t not in priority_terms]
    result = list(set(priority_terms + other_terms[:remaining_slots]))

    logger.info(f"Extracted {len(result)} keywords from query (max:
{max_keywords})")
    return result

def format_opencti_doc(source: Dict, index: str) -> Dict:
    """Convert OpenCTI document format to our expected format"""
    doc_type = "STIX Entity" if "domain" in index else "STIX Relationship"

    # Extract useful fields from the OpenCTI object
    name = source.get('name', source.get('standard_id', 'Unknown'))
    entity_type = source.get('entity_type', source.get('relationship_type',
'Unknown'))
    description = source.get('description', source.get('content',
source.get('summary', '')))
    if not description:
        description = "No description available"

    created = source.get('created_at', source.get('created', 'Unknown date'))

    # Get threat actors from relationship if available
    threat_actors = []
    if 'fromType' in source and source.get('fromType') == 'Threat-Actor':
        threat_actors.append(source.get('fromName', ''))
    if 'toType' in source and source.get('toType') == 'Threat-Actor':
        threat_actors.append(source.get('toName', ''))
```

```
    # Get malware from relationship if available
    malware = []
    if 'fromType' in source and source.get('fromType') == 'Malware':
        malware.append(source.get('fromName', ''))
    if 'toType' in source and source.get('toType') == 'Malware':
        malware.append(source.get('toName', ''))

    # Get TTPs from relationship if available
    ttps = []
    if 'fromType' in source and source.get('fromType') == 'Attack-Pattern':
        ttps.append(source.get('fromName', ''))
    if 'toType' in source and source.get('toType') == 'Attack-Pattern':
        ttps.append(source.get('toName', ''))

    # Create formatted document
    formatted = {
        "report_name": name,
        "source_type": f"OpenCTI {doc_type}",
        "report_published": created,
        "threat_actors": threat_actors,
        "malware": malware,
        "ttps": ttps,
        "text": f"{name}\n\n{description}"
    }

    return formatted

def build_context(documents: List[Dict]) -> str:
    blocks = []
    for i, doc in enumerate(documents):
        blocks.append(f"""### Document {i+1}
**Title:** {doc.get('report_name', 'Untitled')}
**Source:** {doc.get('source_type', 'Unknown')}
**Published:** {doc.get('report_published', 'Unknown')}
**Threat Actors:** {', '.join(doc.get('threat_actors', []))}
**TTPs:** {', '.join(doc.get('ttps', []))}
**Content Preview:**
{doc.get('text', '')[:1500]}...
""")
    return "\n\n".join(blocks)
```

Liite 3 – Cyber Threat Intelligence RAG Assistant -kysely

Enter your threat intelligence query:
You are an advanced Cyber Threat Intelligence analyst. What was the Threat Landscape in 2024 like?

Run Query

Retrieved 50 documents in 0.08 seconds
Answer (generated in 117.27 seconds)

1. Threat Landscape (based on information from context sources):
From reports, it was observed that the threat landscape has shifted towards multi-staged attacks involving advanced techniques such as machine learning, deep learning, and cloud-based threats. These attacks are often coordinated with dark markets for illicit goods or services. Cyber espionage is an increased concern due to the development of advanced attack techniques that mimic human behavior, targeting sensitive information in real time. This includes behaviors like phishing, spearphishing, and data exfiltration. IoT devices have become targets for cyber attacks as they often carry out malicious activities without detection. Cybercriminals are also leveraging smart home security systems to carry out unauthorized access or DDoS attacks. Cloud-based threats, including data breaches and ransomware, continue to be a significant concern due to the growing adoption of cloud services by enterprises.

2. Reported Cyber Espionage:
According to the "Foresight Cybersecurity Threats for 2030", advanced cyber espionage, particularly targeted at US-based institutions, is a growing concern due to the continued expansion of American influence and the rise in advanced attack techniques used for espionage targeting sensitive information within real time.

3. Reported Advanced Attacks:
Reports indicate continued focus on cloud-based attacks with sophisticated threat actors. These might include malware like ransomware but also botnets leveraging advanced mac-

hine learning techniques for reconnaissance and command & control activities within the cloud environment. Cybercriminals are increasingly using deepfake technology to manipulate video content, posing a risk to online education platforms due to its potential misuse by malicious actors.

4. Reported IoT Threats:
An increasing number of IoT devices are being targeted for unauthorized access or data breaches, often without detection due to their scale and sophistication.

In summary, the threat landscape is shifting towards multi-staged attacks using advanced techniques on cloud platforms, targeting sensitive information in real time. Advanced attack techniques are also increasingly used for IoT threats. Cybercriminals are leveraging sophisticated means of espionage, including phishing and credential stuffing.